

REFERENCE EDITION

Contents

1. Slide 1 — Data layer I: authoritative facts and domain separation
2. Slide 2 — Data layer II: atomic writes, events, and graph projection
3. Slide 3 — Capability 00: deterministic POC dataset
4. Slide 4 — Capability 01: Neo4j projection
5. Slide 5 — Capability 02: temporal and graph feature layer
6. Slide 6 — Capability 03: event memory
7. Slide 7 — Capability 04: behaviour intelligence
8. Slide 8 — Capability 05: churn prediction
9. Slide 9 — Capability 06: recommendations and uncertainty
10. Slide 10 — Capability 07: graph fraud
11. Slide 11 — Capability 08: SFA forecasting
12. Slide 12 — Capability 09: computed digital twins
13. Slide 13 — Capability 10: decision engine and explanations
14. Slide 14 — Capability 11: grounded Copilot
15. Cross-capability technical themes
16. Honest POC limitations
17. Questions to prepare for
18. Recommended closing

Telco Digital — Technical Presentation Notes v1

This document is the detailed technical narrative for presenting the Telco Digital shared-intelligence POC. It follows the 14-slide structure in `ppt_content_v2.md` and focuses on architecture, data, capability behaviour, technical reasoning, evidence, and limitations rather than source-code explanation.

Presentation objective

The goal is to demonstrate that the work is more than a collection of isolated models. It is an end-to-end intelligence platform that connects:

```
Operational facts
→ historical and relationship context
→ reusable features
→ intelligence capabilities
→ governed decisions
→ explanations and presentation
```

The central technical argument is:

Intelligence is trustworthy only when its source facts, time boundaries, relationships, model versions, uncertainty, and business rules are explicit.

Recommended opening

"I built a shared-intelligence POC for telecom and retail use cases. PostgreSQL stores authoritative facts and history, Neo4j provides rebuildable relationship context, temporal services reconstruct the customer at a selected time, and several intelligence capabilities reuse those contracts. Predictions do not directly execute actions; a separate decision layer applies business rules and produces explanations."

Core terminology

Term	Meaning in this platform
Fact	An observed business event or state recorded by the platform
Ledger	Append-only record of value changes used for reconstruction
Event	Immutable statement that something happened
Feature	A calculated value derived from facts at a selected time
Inference	Interpreted behaviour or trait derived from evidence
Prediction	A model estimate such as churn probability
Recommendation	A ranked valid option from the product catalogue
Decision	A governed action selected under rules and uncertainty
Explanation	The reason, evidence, alternatives, confidence, and unknowns
Digital twin	A computed point-in-time view combining facts and intelligence

Flowchart legend

The diagrams use the same visual meaning throughout the document:

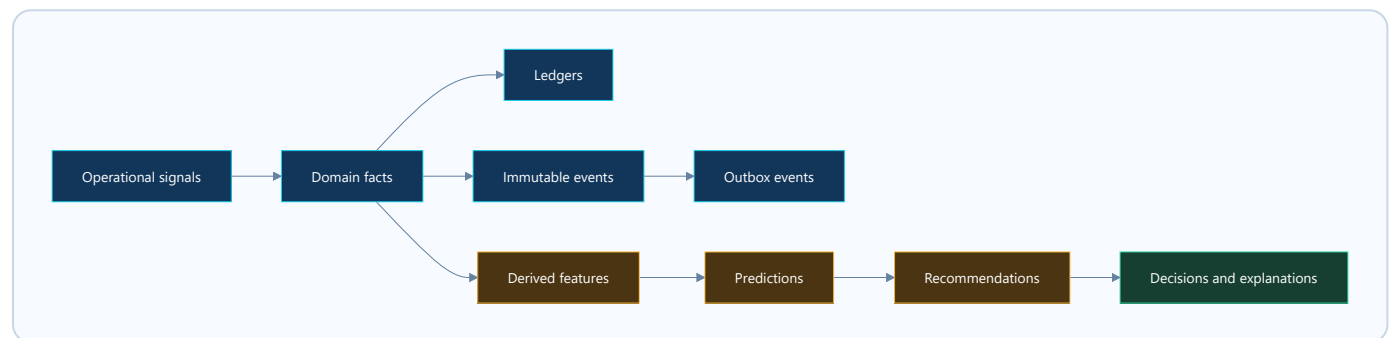
- Blue: authoritative facts and operational data
 - Purple: Neo4j relationship context
 - Amber: derived features, intelligence, or predictions
 - Green: governed decisions and presentation outputs
 - Gray: unknown or unavailable information
-

Slide 1 — Data layer I: authoritative facts and domain separation

Core message

PostgreSQL is the authoritative source of truth. The platform separates domain facts, event history, asynchronous integration, and derived intelligence so that each type of information has a clear meaning and lifecycle.

Explanation flowchart

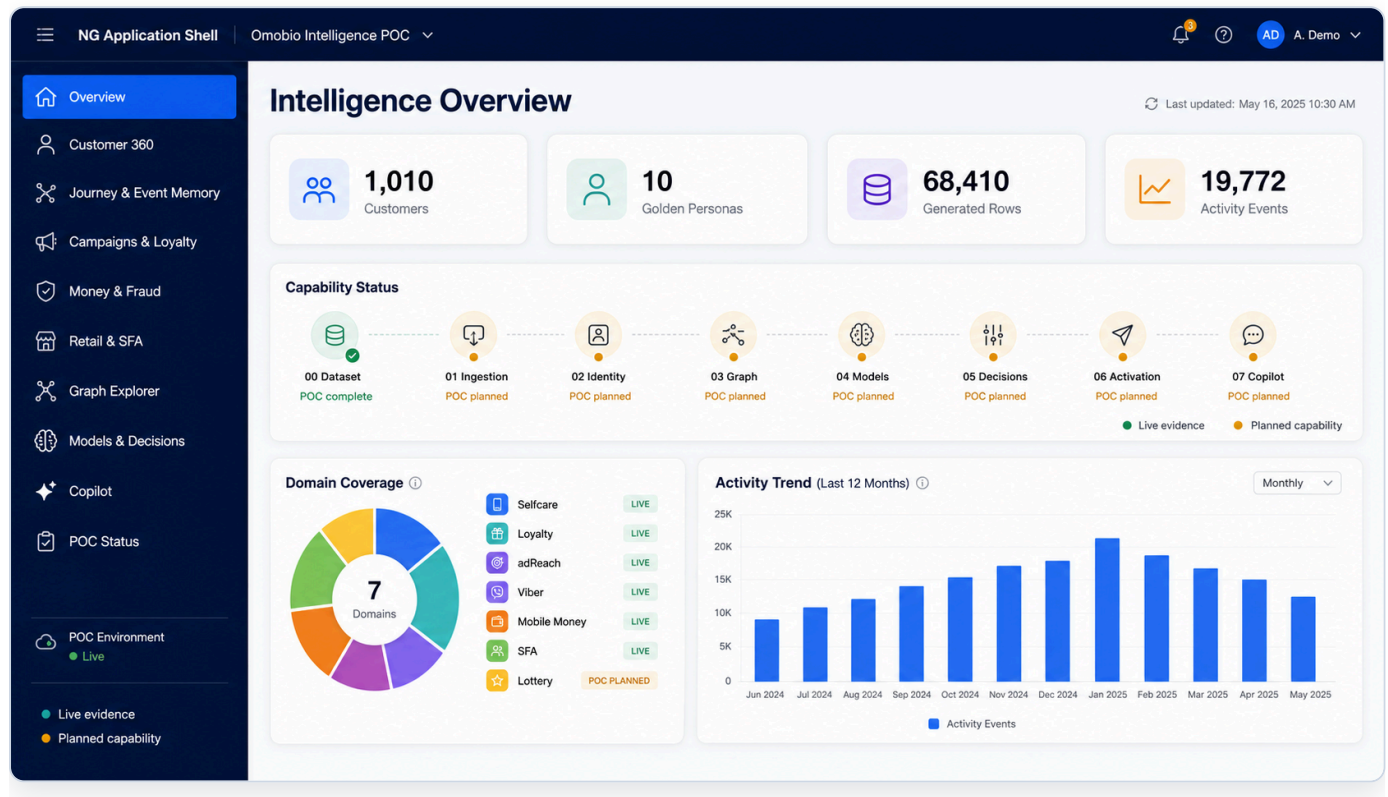


Presentation visual



Use this visual to explain the complete journey from operational signals and authoritative databases to temporal context, graph relationships, intelligence modules, a computed twin, governed decisions, and the final application experience.

Supporting live UI evidence:



Technical details to present

Domain separation

The database is divided into logical schemas:

- **core** contains stable entities such as customers, accounts, SIMs, devices, plans, and subscriptions.
- **telco** contains usage, recharge, travel, service interactions, and balance-ledger activity.
- **money** contains wallets, merchants, and money transactions.
- **marketing** contains loyalty and campaign signals.
- **sfa** contains retailers, products, sales, inventory, and promotion-related facts.
- **activity** contains immutable cross-domain events.
- **integration** contains outbox events used for reliable graph projection.
- **intelligence** contains derived outputs such as feature snapshots, predictions, recommendations, and warnings.

Why schema separation matters

The separation prevents ambiguity. A recharge is an observed business fact. A churn probability is a model output. A recommendation is a proposed action. If these records are mixed together, the system cannot clearly answer which values are authoritative, which can be recalculated, and which depend on a model version.

Ledger design

A current balance column answers only “what is the balance now?” A ledger answers:

- What changed the balance?
- When did the change happen?
- What was the balance at an earlier time?
- Which transaction was responsible?

Historical balance is reconstructed by summing ledger entries up to the selected time.

Source-of-truth rule

PostgreSQL remains authoritative even when Neo4j or a model service is unavailable. Relationship projections and intelligence outputs can be rebuilt or recalculated from PostgreSQL facts.

Derived-intelligence rule

Features, inferences, predictions, and recommendations are stored separately because they can change when:

- The selected `as_of` changes
- A feature definition changes
- A model version changes
- A catalogue changes
- New evidence becomes available

Digital-twin rule

The digital twin is computed from the current contracts. It is not treated as a master customer record because that would duplicate authority and eventually become stale.

Strong interview statement

“I designed the data layer so that for any prediction or decision, we can identify the source facts, the selected time, the feature version, and the reasoning that produced the result.”

Likely follow-up question

Why not store everything in one customer table?

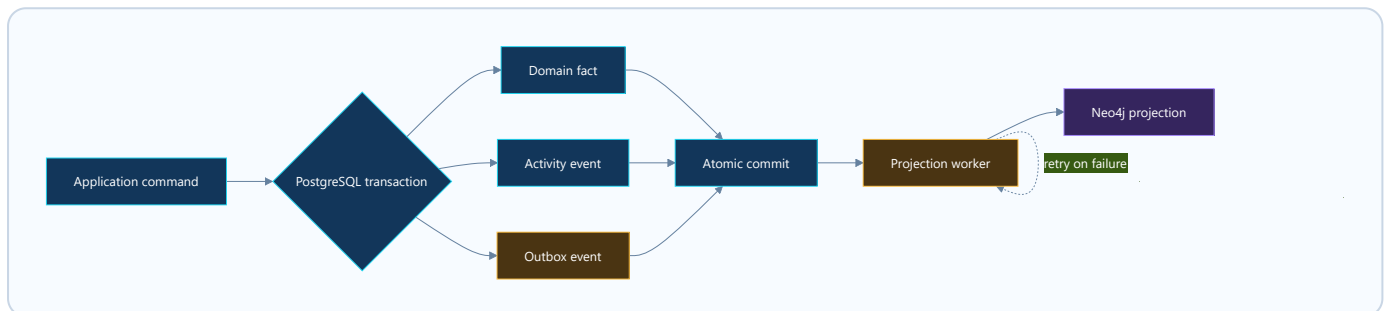
Because current state, event history, model outputs, and recommendations have different consistency and lifecycle requirements. Combining them would make historical reconstruction, model auditing, and correction difficult.

Slide 2 — Data layer II: atomic writes, events, and graph projection

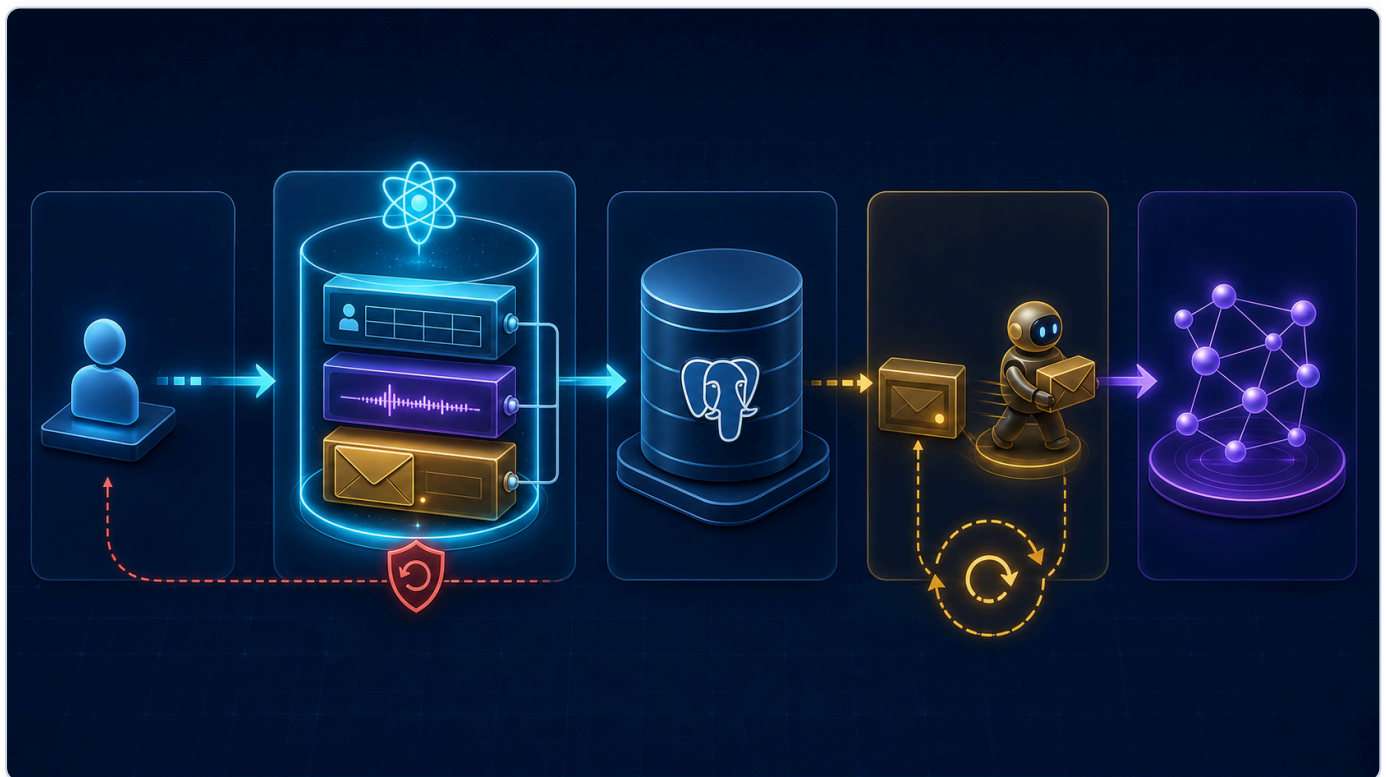
Core message

Important operations are written atomically to PostgreSQL, while Neo4j is updated asynchronously through a transactional outbox.

Explanation flowchart



Presentation visual



Suggested editable labels for the final slide:

Command · Atomic transaction · PostgreSQL commit · Outbox worker · Neo4j projection

Technical details to present

Atomic write path

One PostgreSQL transaction contains:

1. The domain fact
2. The immutable activity event
3. The outbox event

For example, a plan purchase can create:

- A subscription fact
- A balance-ledger entry
- A `PLAN_PURCHASED` activity event
- A graph-projection outbox event

If any part fails, the full transaction is rolled back.

Why direct dual writes are unsafe

Writing to PostgreSQL and Neo4j during the same API request creates partial-success risk:

- PostgreSQL succeeds and Neo4j fails
- Neo4j succeeds and PostgreSQL fails
- The request times out after one side commits

The outbox changes the problem. PostgreSQL commits the business operation and a durable delivery instruction together. A separate worker can retry graph projection safely.

Event timestamps

The event model keeps two timestamps:

- `occurred_at` : when the business activity happened
- `recorded_at` : when this platform received or stored it

This supports delayed and out-of-order events.

Projection worker

The worker:

1. Claims pending outbox records
2. Loads the authoritative graph snapshot
3. Projects managed nodes and relationships
4. Reconciles source and graph counts
5. Marks the batch processed only after success

Idempotency

Graph writes use `MERGE`-style semantics. Replaying the same projection does not create duplicate managed nodes or relationships.

Rebuildability

Neo4j can be cleared for the managed projection and rebuilt from PostgreSQL. This is a powerful recovery property because graph corruption or projection failure does not destroy authoritative business data.

POC trade-off

The POC uses a controlled full-snapshot rebuild. This is operationally simple and easy to reconcile, but a production platform would normally move toward incremental event-level projection, distributed workers, exponential retry, dead-letter queues, lag monitoring, and alerting.

Strong interview statement

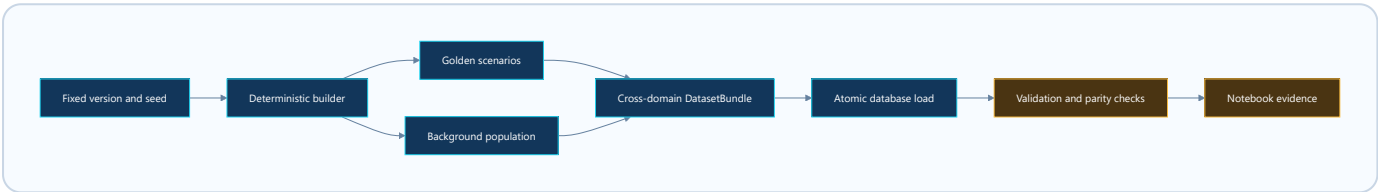
"I did not try to create a distributed transaction across PostgreSQL and Neo4j. I used the transactional-outbox pattern so the business write remains correct and graph delivery becomes retryable."

Slide 3 — Capability 00: deterministic POC dataset

Core message

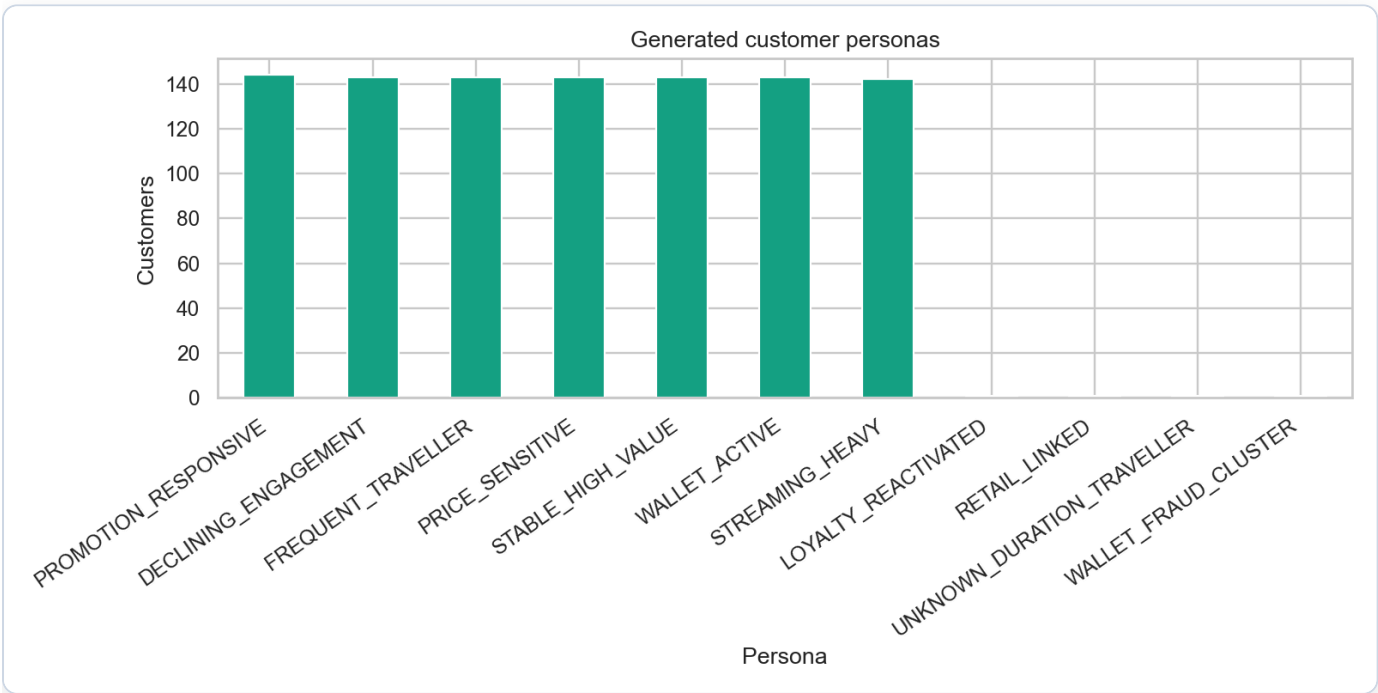
The POC uses reproducible synthetic data that covers multiple business domains and provides controlled scenarios for every later capability.

Explanation flowchart



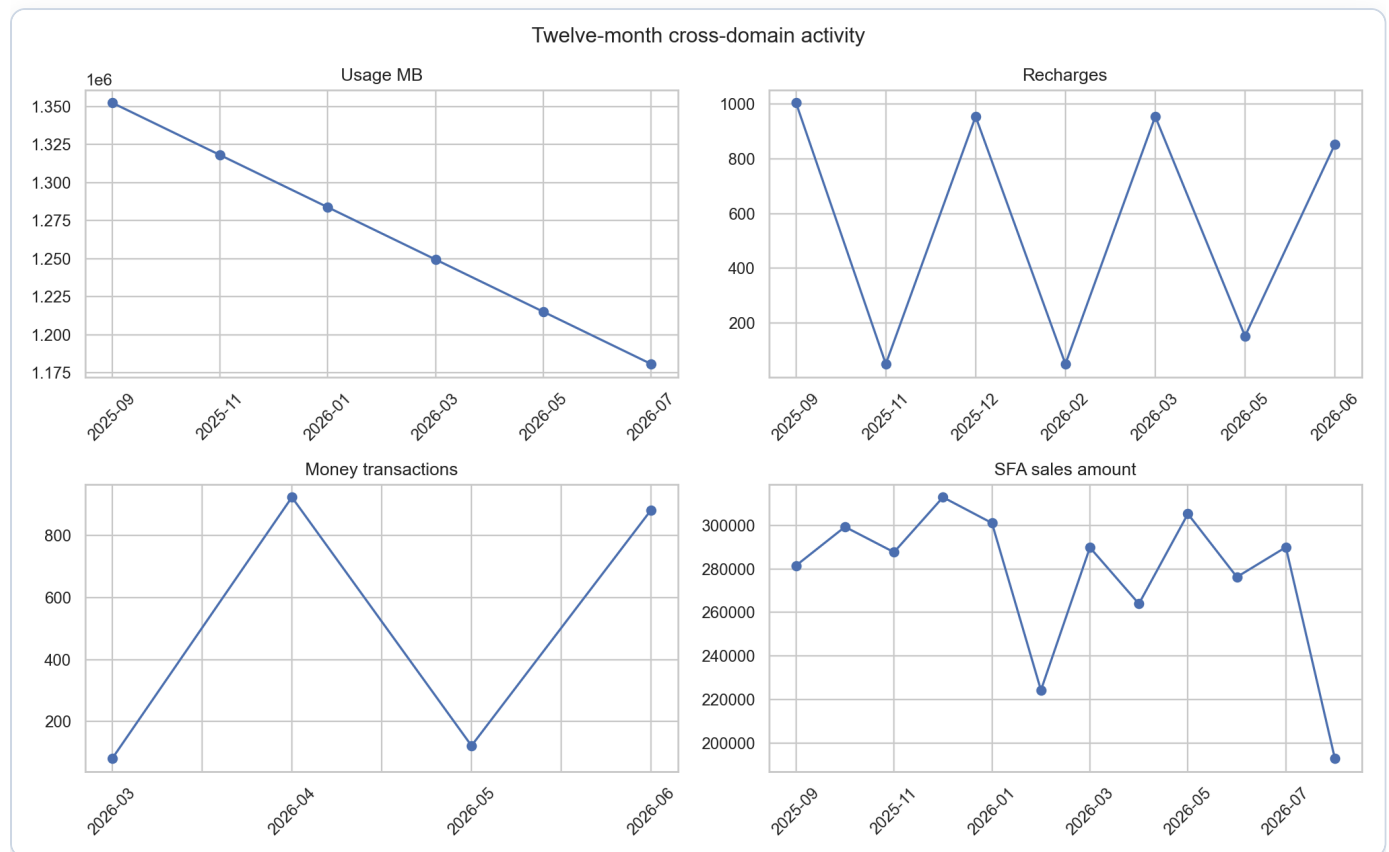
Notebook evidence

Generated persona distribution



The background personas are intentionally balanced so later notebooks can compare known behavioural scenarios without severe class imbalance.

Cross-domain activity over time



Supporting artifacts

- [Dataset metrics](#)
- [Table row counts](#)
- [Persona distribution data](#)

Technical details to present

Why the dataset was necessary

The intelligence capabilities require correlated signals, not isolated random rows. For example:

- Churn needs declining usage and engagement patterns.
- Event memory needs repeated travel episodes.
- Recommendations need historical trip duration, plan, and usage.
- Graph fraud needs shared devices, wallets, merchants, and suspicious paths.
- SFA forecasting needs retailer sales and inventory history.

Dataset composition

The dataset contains:

- Golden customers with named behaviours and expected outcomes
- 1,000 background customers for population context
- Telecom, loyalty, marketing, mobile-money, service, and SFA records

- Activity events and matching outbox records
- Time-ordered data covering a controlled one-year period

Determinism

The generator uses a fixed dataset version and seed. Repeated generation produces the same scenario structure, enabling reproducible tests, notebook outputs, and demonstrations.

Evidence

- 1,005 generated customers in the expanded dataset
- 19,772 activity events
- 19,772 outbox events
- Event/outbox parity is true
- 6,030 usage events
- 4,020 recharges
- 2,010 money transactions
- 1,200 sales records
- Dataset validation is true

Important limitation

The population is scenario-shaped. It demonstrates the technical workflow and expected behaviours but does not represent real customer distributions or prove model accuracy.

Strong interview statement

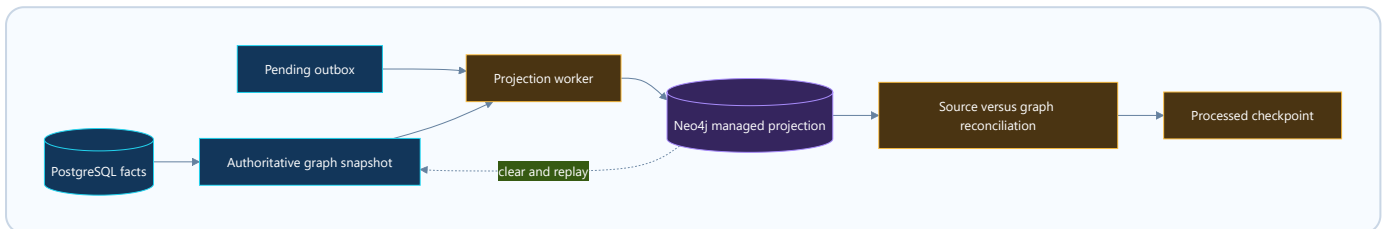
“The dataset is synthetic, but it is not arbitrary. It is deterministic and designed to exercise cross-domain scenarios with known expected results.”

Slide 4 — Capability 01: Neo4j projection

Core message

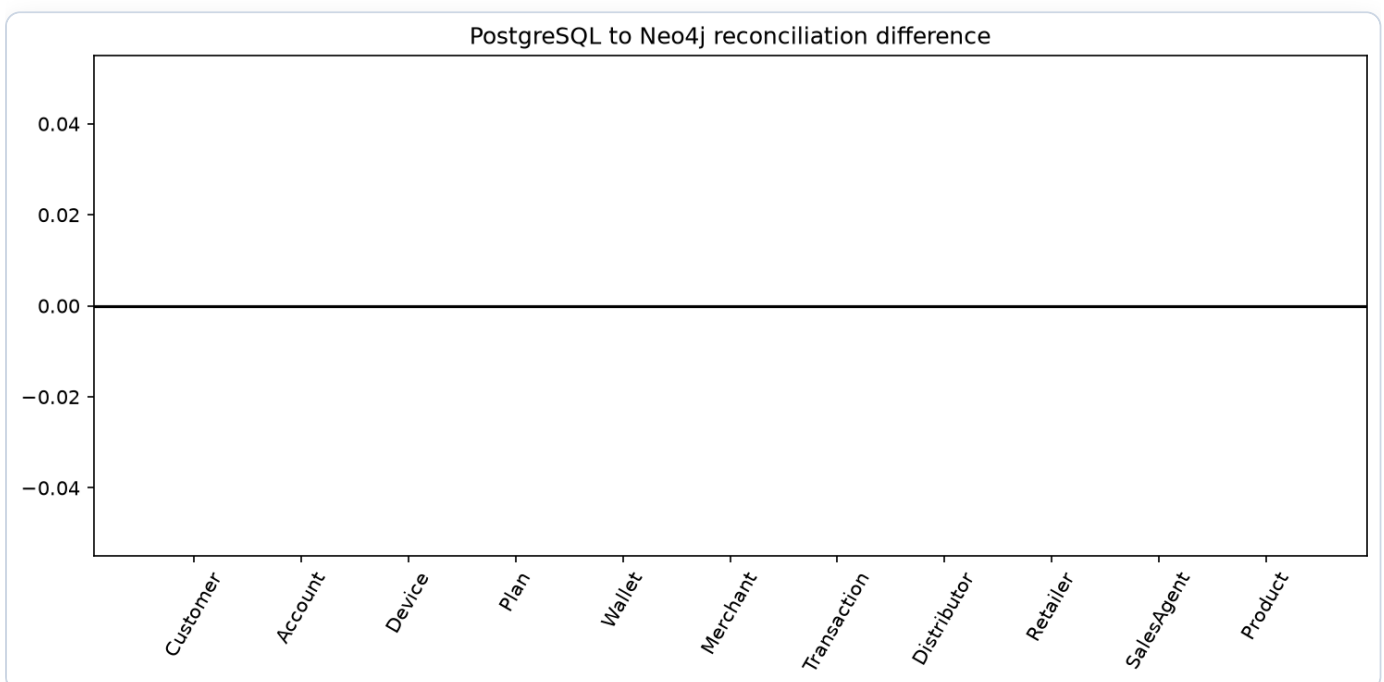
Neo4j stores rebuildable relationship context, not authoritative customer state or changing predictions.

Explanation flowchart



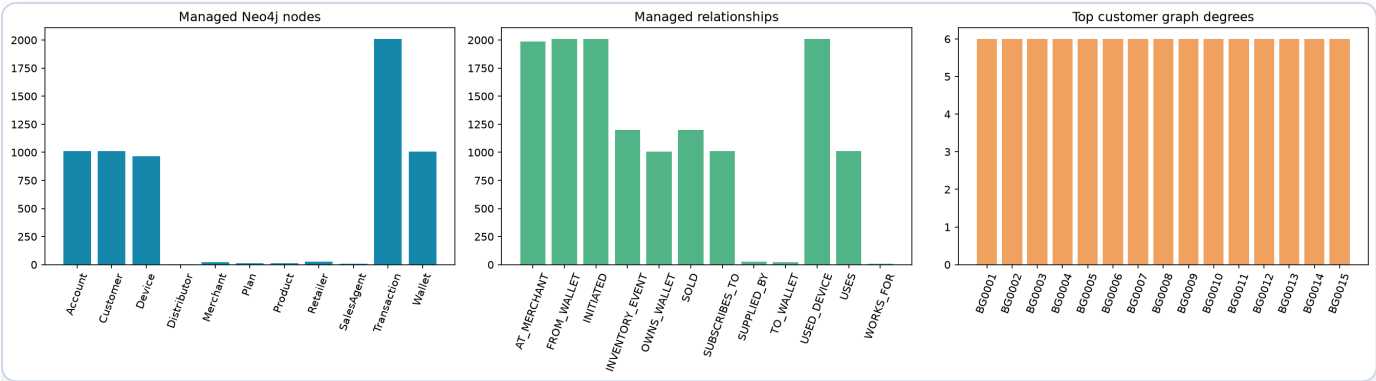
Notebook and UI evidence

Source-to-projection reconciliation

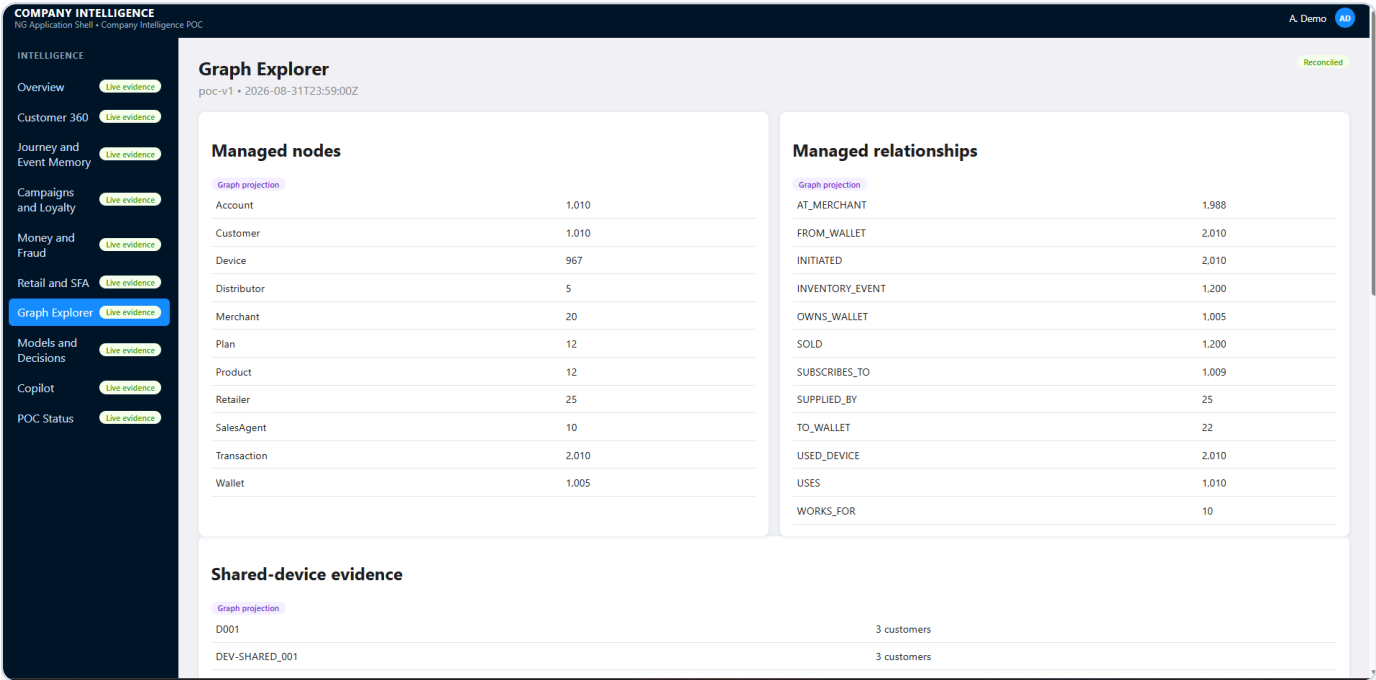


The zero line is the important result: the compared source and projected entity counts have no difference.

Projected graph composition



Live Graph Explorer



Supporting artifacts

- Projection metrics
- Reconciliation table
- Shared-device evidence

Technical details to present

Relationship model

The projection includes entities such as:

- Customer
- Account
- Device
- Plan

- Wallet
- Merchant
- Transaction
- Distributor
- Retailer
- Sales agent
- Product

Relationships represent observed connections, including:

- Customer uses device
- Customer owns wallet
- Customer subscribes to plan
- Transaction originates from wallet
- Transaction goes to wallet or merchant
- Retailer is supplied by distributor
- Sales agent works for an organisation

Why Neo4j is a projection

Graph databases are useful for traversal, but PostgreSQL provides the primary transaction and history guarantees needed by the business system. Keeping the graph rebuildable avoids two competing sources of truth.

Reconciliation

After rebuilding the graph, the system compares source entities and projected node counts. The notebook evidence shows zero reconciliation difference for the compared entity categories.

Evidence

- 1,010 customer nodes
- 1,010 account nodes
- 967 device nodes
- 1,005 wallet nodes
- 2,010 transaction nodes
- 22 shared-device cases
- Maximum customer degree: 6
- Projection reconciled: true

Important limitation

The worker is a controlled single-worker POC. It does not demonstrate production throughput, distributed locking, or continuous projection-lag monitoring.

Strong interview statement

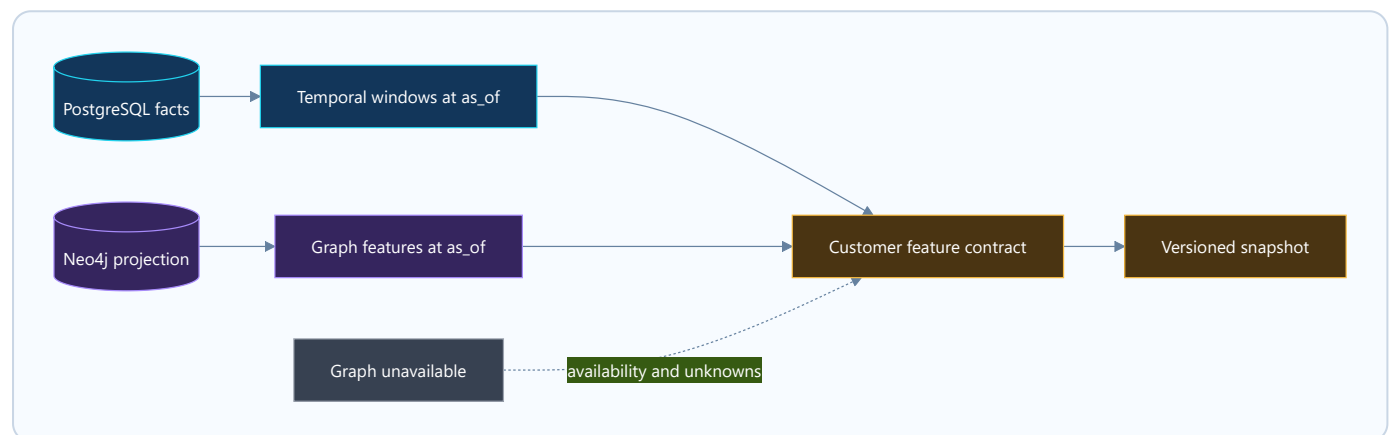
"Neo4j gives us relationship traversal, but PostgreSQL still owns the facts. If the graph disappears, we can rebuild it and prove that it matches the source."

Slide 5 — Capability 02: temporal and graph feature layer

Core message

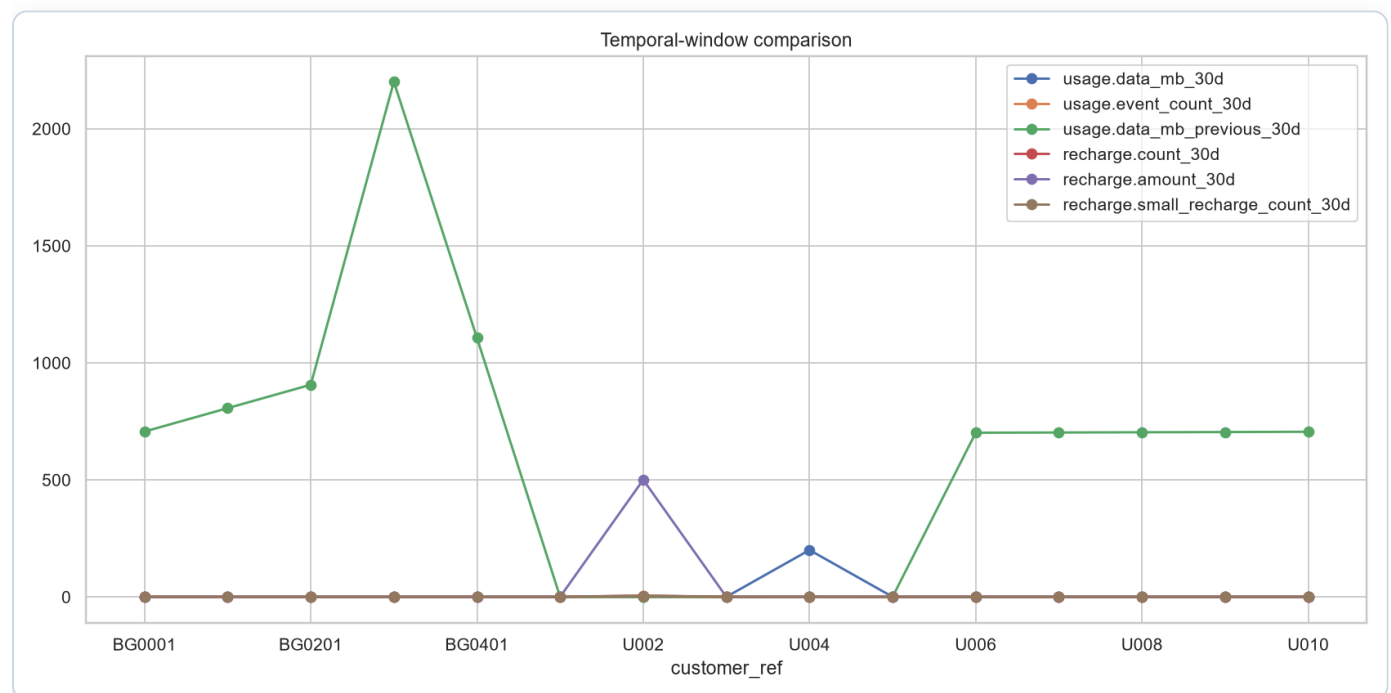
A single versioned feature contract combines bounded relational and graph evidence for a customer at a selected time.

Explanation flowchart



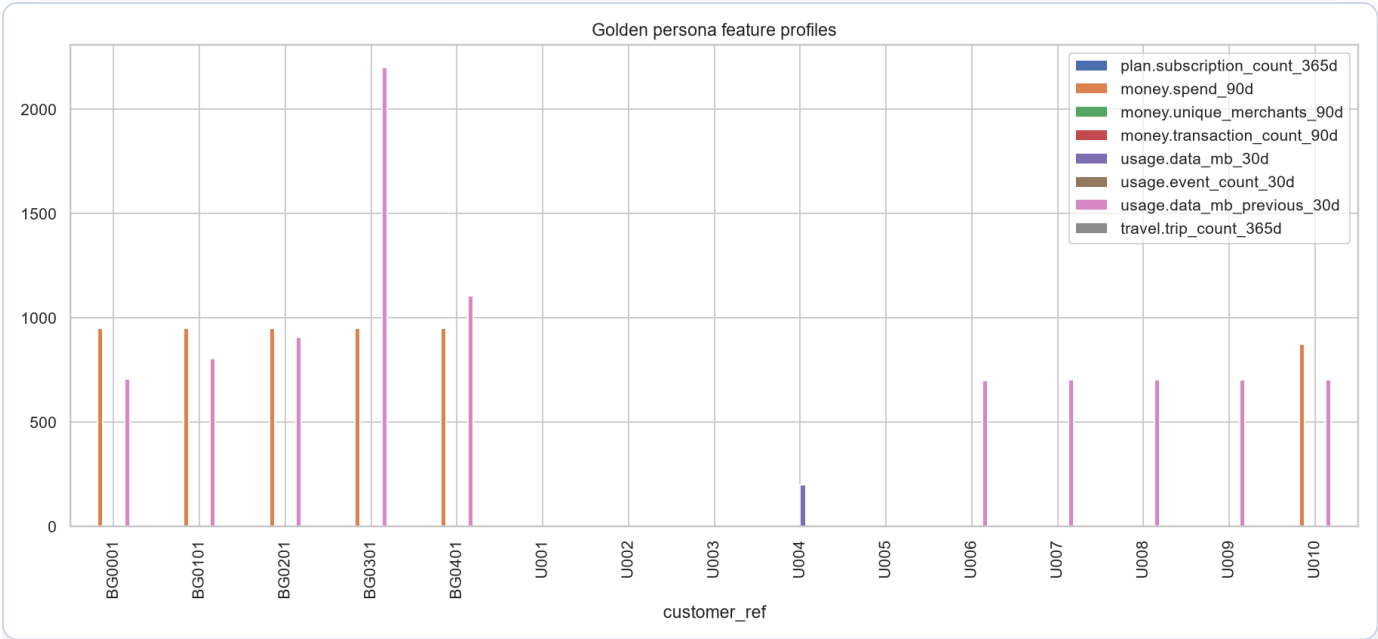
Notebook evidence

Temporal-window comparison

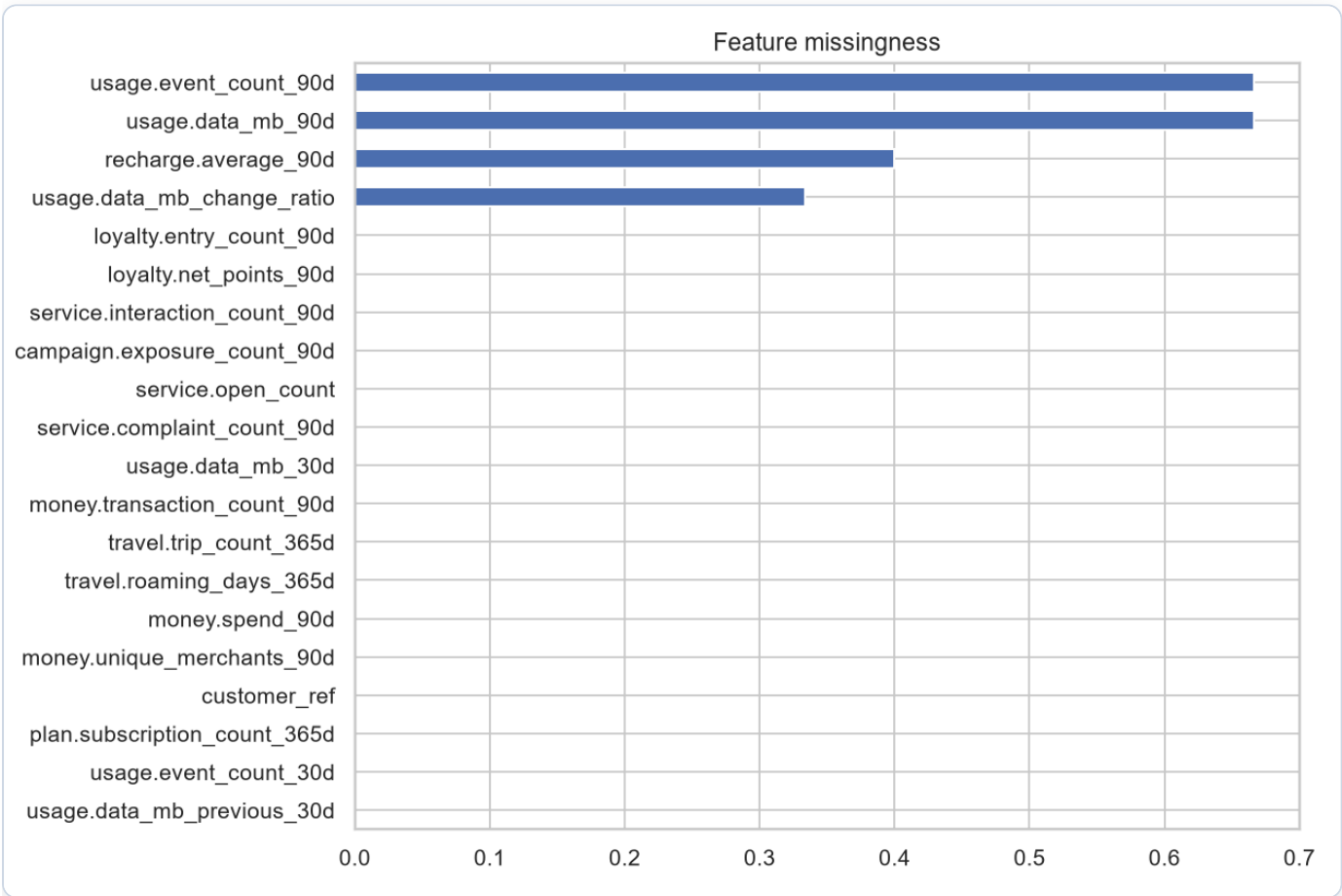


This figure demonstrates that current and previous windows are distinct feature inputs rather than one mutable customer total.

Feature profiles by persona



Missingness and unknowns



Supporting artifacts

- Feature metrics
- Feature profiles
- Future-leakage validation
- Missingness table

Technical details to present

Temporal features

Temporal features are calculated from PostgreSQL facts using time windows such as:

- Last 30 days
- Last 90 days
- Previous 30-day window
- Change between current and previous windows

Feature groups include:

- Usage
- Recharge
- Money
- Plan
- Travel
- Service
- Loyalty
- Campaign

Graph features

Relationship features include:

- Customer degree
- Shared-device context
- Counterparty counts
- Merchant relationships
- Transaction neighbourhood context

Graph relationships are bounded by the same `as_of` principle where temporal relationship properties allow it.

Feature contract

The contract version is `customer-features-v1`. Versioning ensures downstream models and decisions know which definitions produced the values.

Deterministic snapshots

The snapshot identity is deterministically derived from:

- Customer identity
- `as_of`
- Feature version

Materializing the same snapshot again updates the same logical record rather than creating unrelated duplicates.

Read versus materialization

Feature retrieval is read-only. Persistence happens only through an explicit materialization process, preventing ordinary query calls from causing hidden writes.

Unknown handling

If graph context is unavailable, the contract reports:

- Availability status
- Provenance
- Reason for unavailability
- Unknown graph values

It does not fabricate zeros.

Evidence

- 15 feature snapshots
- 29 numeric features
- Graph available for all 15 notebook snapshots
- Future-leakage checks failed: 0

Strong interview statement

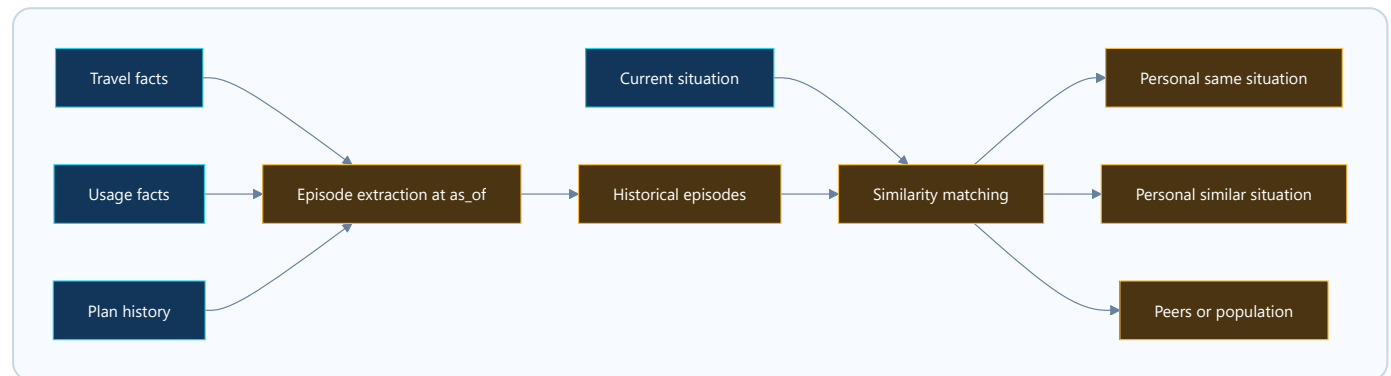
“The feature layer is the reusable boundary between raw operational data and intelligence. Models do not independently query business tables and redefine features.”

Slide 6 — Capability 03: event memory

Core message

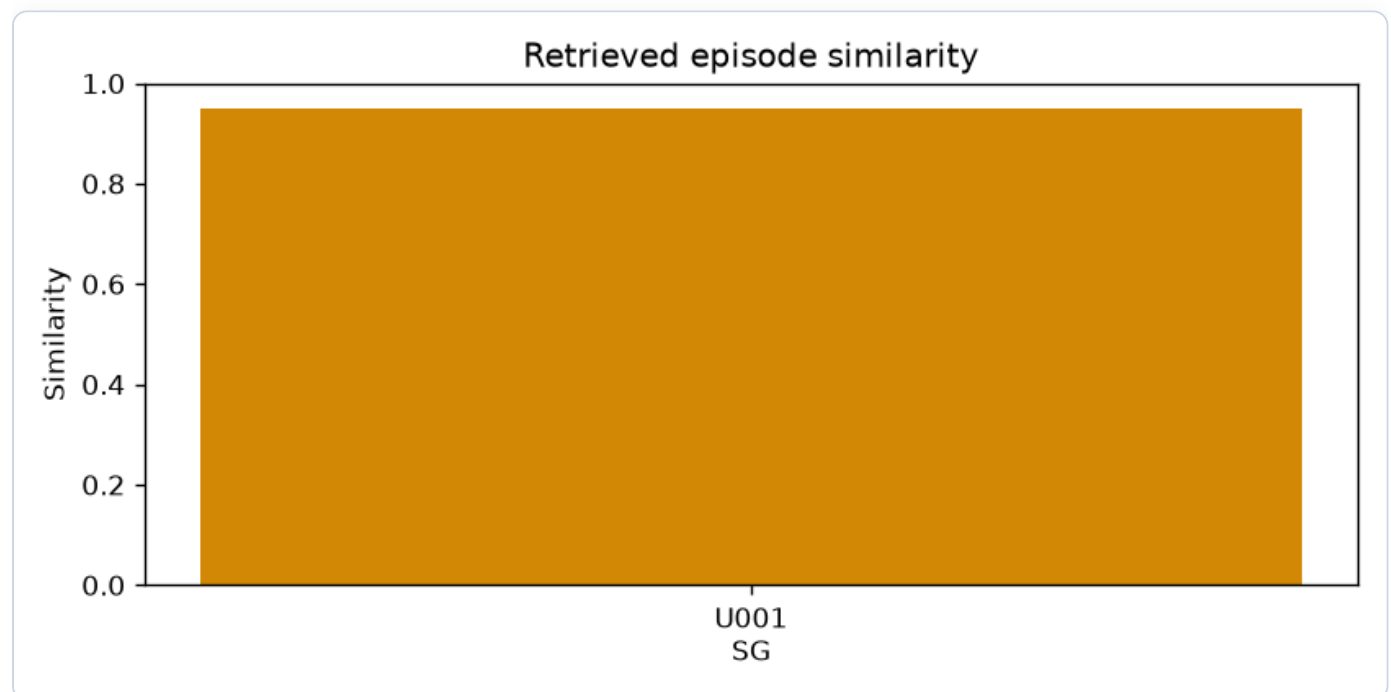
Event memory reconstructs historical travel episodes and retrieves the most relevant previous experience for the current situation.

Explanation flowchart

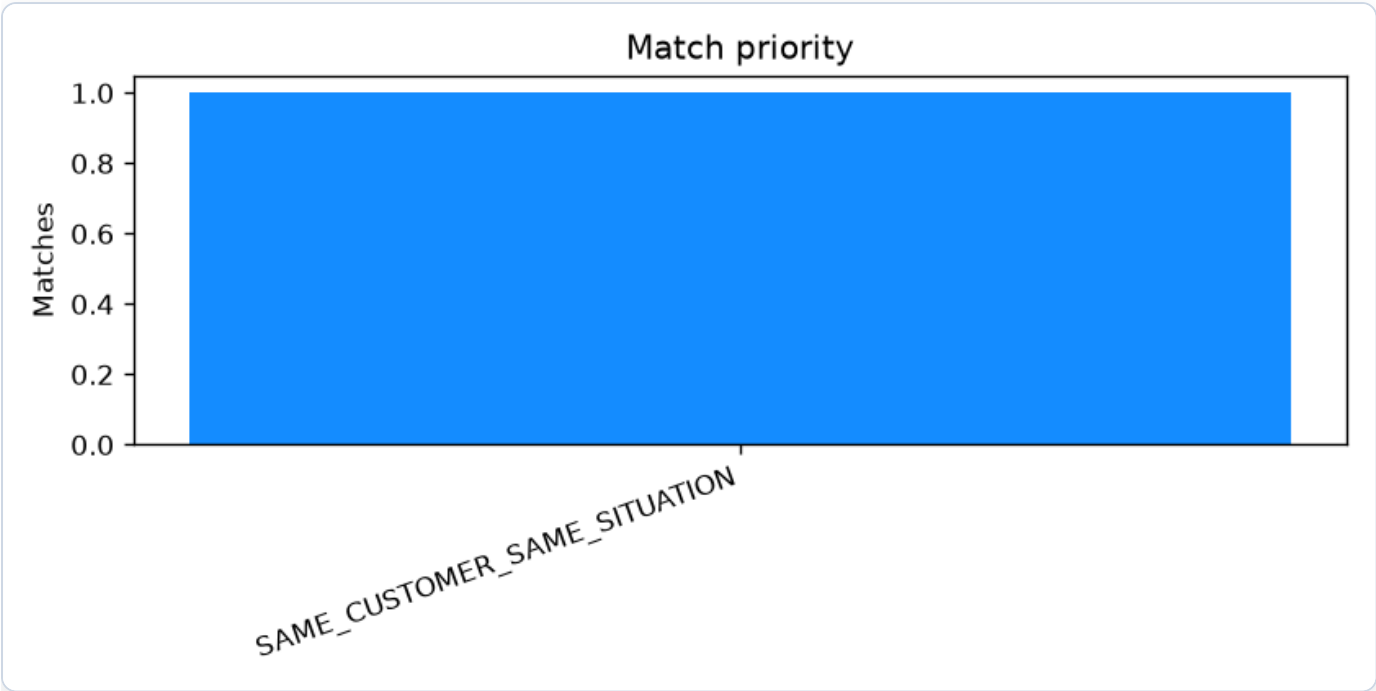


Notebook and UI evidence

Episode similarity



Retrieval priority



Live journey and memory view

COMPANY INTELLIGENCE

NO Application Shell • Company Intelligence POC

INTELLIGENCE

Overview

Customer 360

Journey and Event Memory

Campaigns and Loyalty

Money and Fraud

Retail and SFA

Graph Explorer

Models and Decisions

Copilot

POC Status

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Journey and Event Memory

Derived travel episodes, similar-event matches, catalogue offers and a next-best action.

Golden persona

As of date (optional)

Query destination

Recall episodes

Frequent Traveller — U001

mm/dd/yyyy

SG

Current situation

Derived

No current travel situation at this as_of.

Destination Known

No

Duration Known

Unknown

travel-episodes-v1 • Source: derived_live • as_of 31 Aug 2026, 23:59 UTC

Unknown Current destination is unknown; matching used historical travel only.

Retrieved episodes

Derived

SAME CUSTOMER SIMILAR SITUATION

Singapore travel episode

U001 • 10 Mar 2026, 08:00 UTC

Similarity

0.4

Duration

6 days

Usage

11.4 GB

Plan Selected

ROAM_15

POPULATION

Singapore travel episode

BG0274 • 7 Feb 2026, 00:00 UTC

Similarity

0.05

Duration

6 days

Usage

0 GB

Plan Selected

None

Supporting artifacts

- Event-memory metrics
- Historical U001 episode
- Match ranks
- Future-leakage validation

Technical details to present

Episode construction

A travel episode combines:

- Destination
- Start time
- Effective end time
- Duration when known
- Data usage during the trip
- Active roaming plan
- Outcome derived from the observed facts

Point-in-time correctness

If a trip ends after the selected `as_of`, the future end time is not used. Duration remains unknown at that historical point.

Retrieval hierarchy

Matches are ranked in this order:

1. Same customer and same situation
2. Same customer and similar situation
3. Similar customers
4. Population history

This avoids using population averages when stronger personal evidence is available.

Evidence scenario

For U001:

- Historical Singapore trip duration: 6 days
- Historical usage: 11.4 GB
- Historical plan: `ROAM_15`
- Recorded outcome: no additional package required
- New situation top rank: same customer, same situation
- Similarity: 0.95
- Future-leakage checks failed: 0

Current limitation

Episodes are reconstructed from facts rather than stored in a durable event-memory database. Retrieval uses deterministic similarity rather than embeddings or learned retrieval.

Strong interview statement

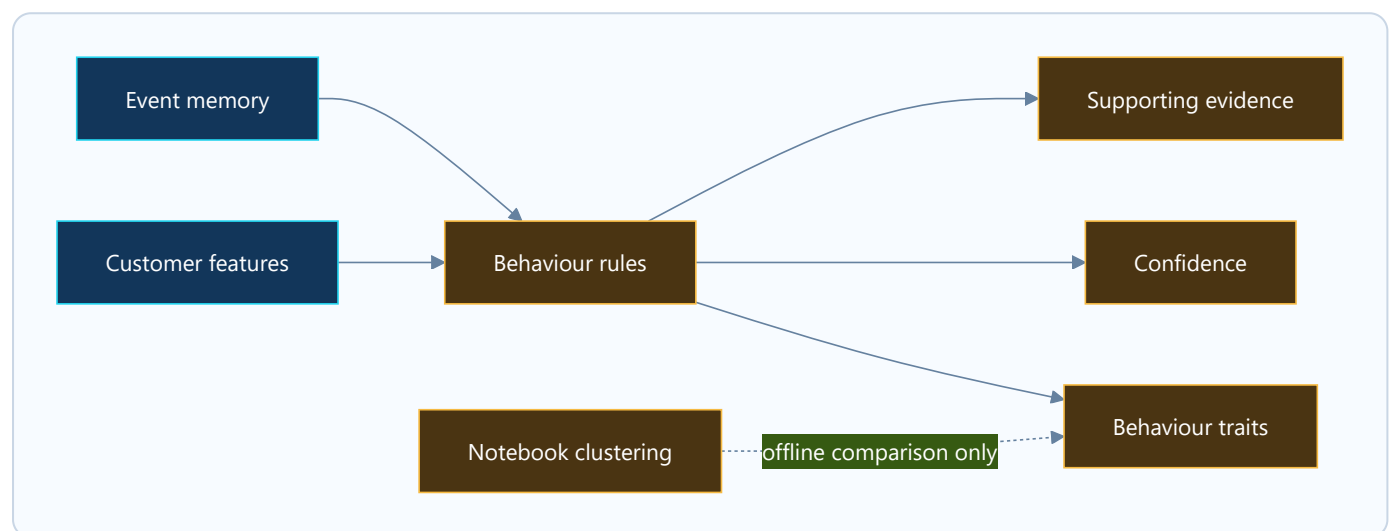
"The recommendation can use the customer's own previous experience before making assumptions from other customers."

Slide 7 — Capability 04: behaviour intelligence

Core message

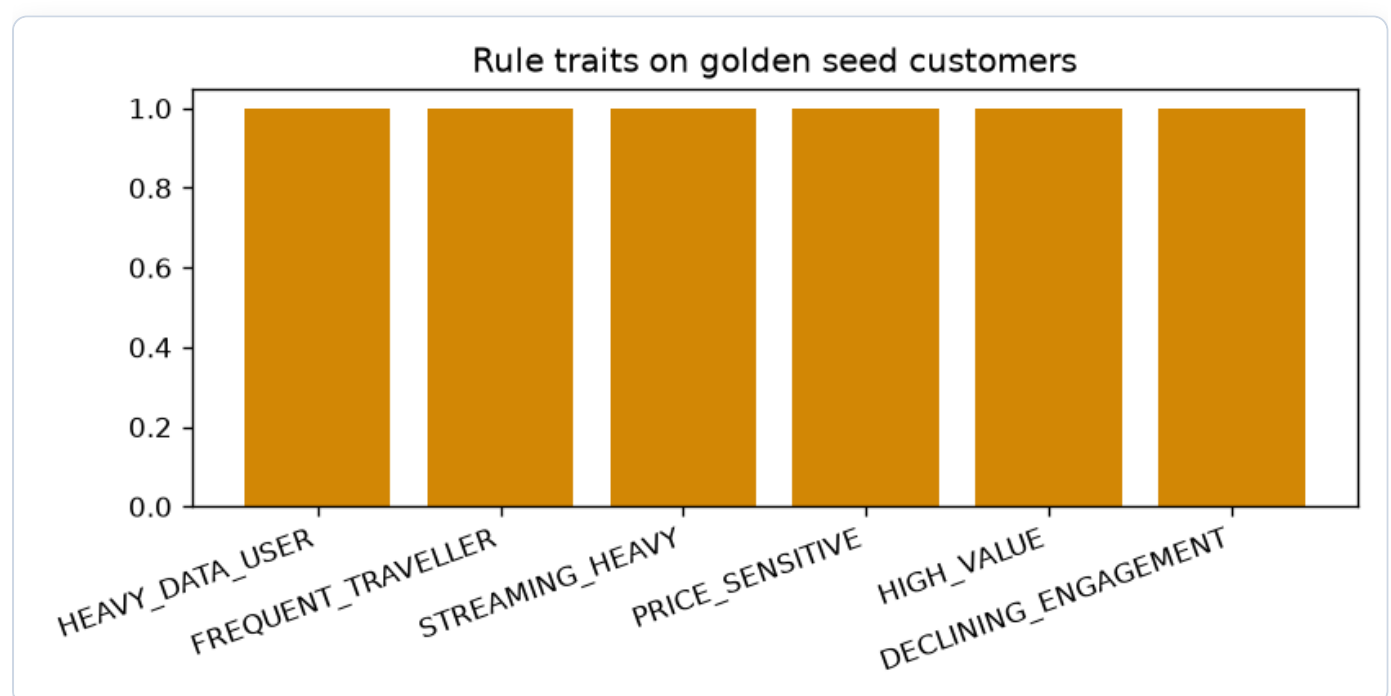
Behaviour traits are explainable inferences derived from features and episodes at a selected time.

Explanation flowchart

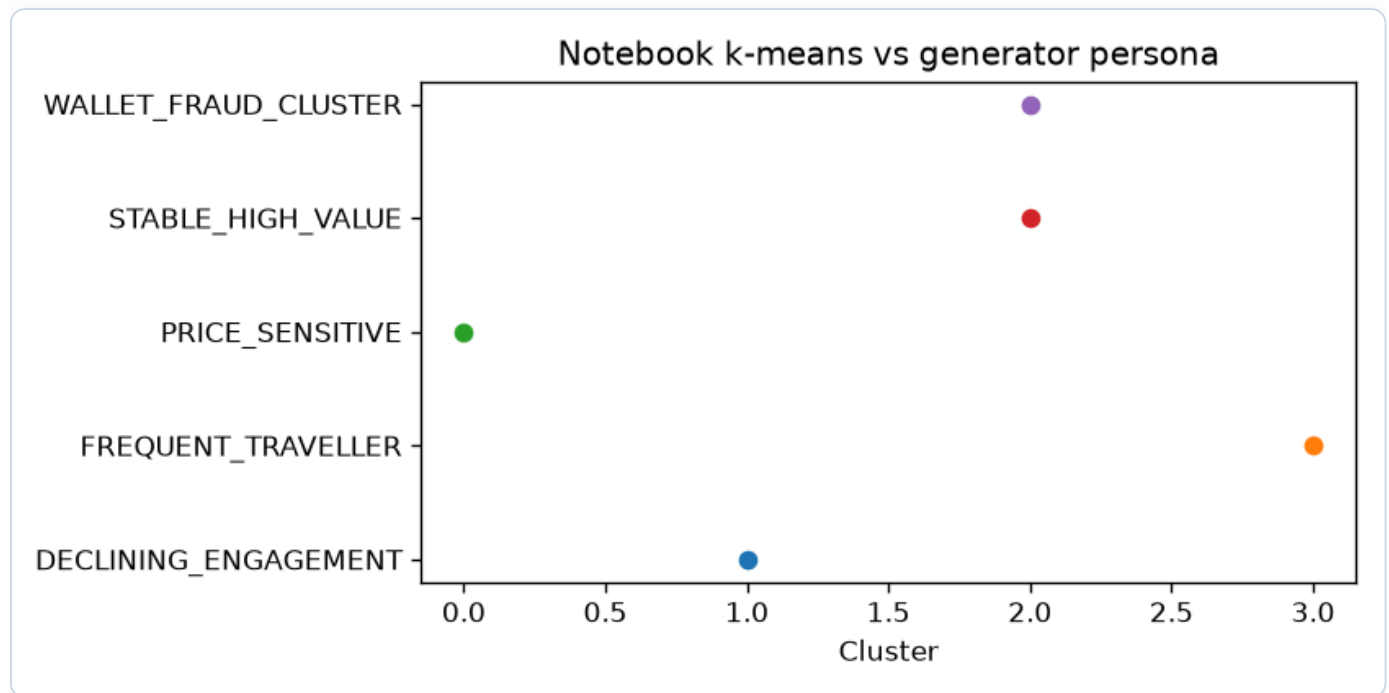


Notebook evidence

Deterministic behaviour traits



Offline cluster comparison



The second figure is analysis evidence only; the served behaviour contract remains deterministic and evidence-backed.

Supporting artifacts

- Behaviour metrics
- Golden customer traits
- Price-sensitive U002 evidence
- Cluster comparison data

Technical details to present

Behaviour versus fact

`FREQUENT_TRAVELLER` is not an immutable customer fact. It is an inference supported by observed travel history. If evidence or thresholds change, the trait can change.

Online behaviour logic

The runtime uses deterministic rules. Each trait includes:

- Trait name
- Confidence
- Supporting evidence
- Version and point-in-time context

Offline clustering

Notebook clustering compares unsupervised groups with the personas used by the synthetic generator. It is analysis evidence, not an online model loaded by the application.

Why deterministic traits are useful

- Easy to explain
- Easy to test
- Stable for POC demonstration
- Clear relationship between evidence and label
- Useful inputs for recommendations and decisions

Evidence

- Behaviour version: `customer-behaviour-v1`
- 6 supported traits
- U001: frequent traveller
- U002: price sensitive
- 4 offline notebook clusters
- Online clustering: false

Limitation

Thresholds are demonstrative and require validation against real customer outcomes before production use.

Strong interview statement

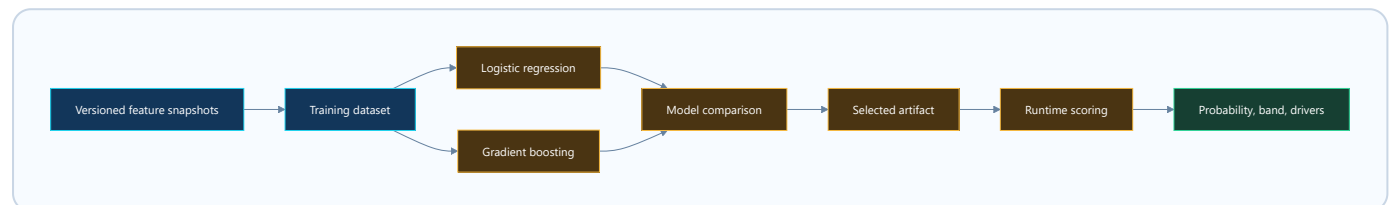
"I kept the served traits deterministic and evidence-backed. Clustering was used to analyse the population, not silently converted into customer truth."

Slide 8 — Capability 05: churn prediction

Core message

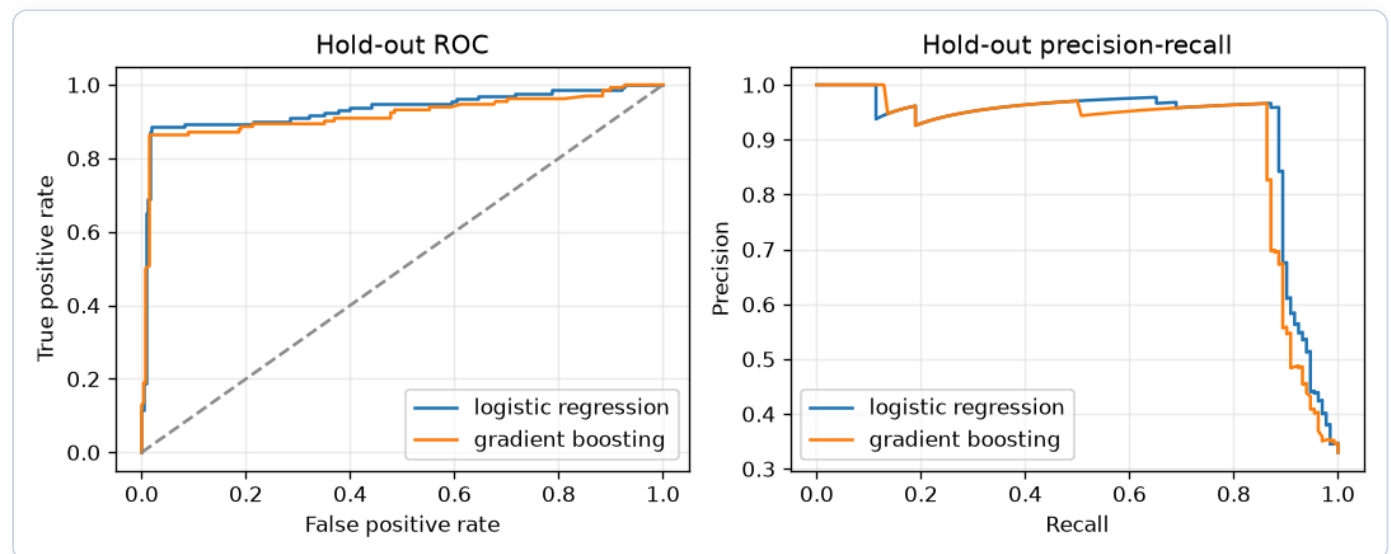
The churn capability separates offline model comparison from lightweight, versioned runtime scoring.

Explanation flowchart

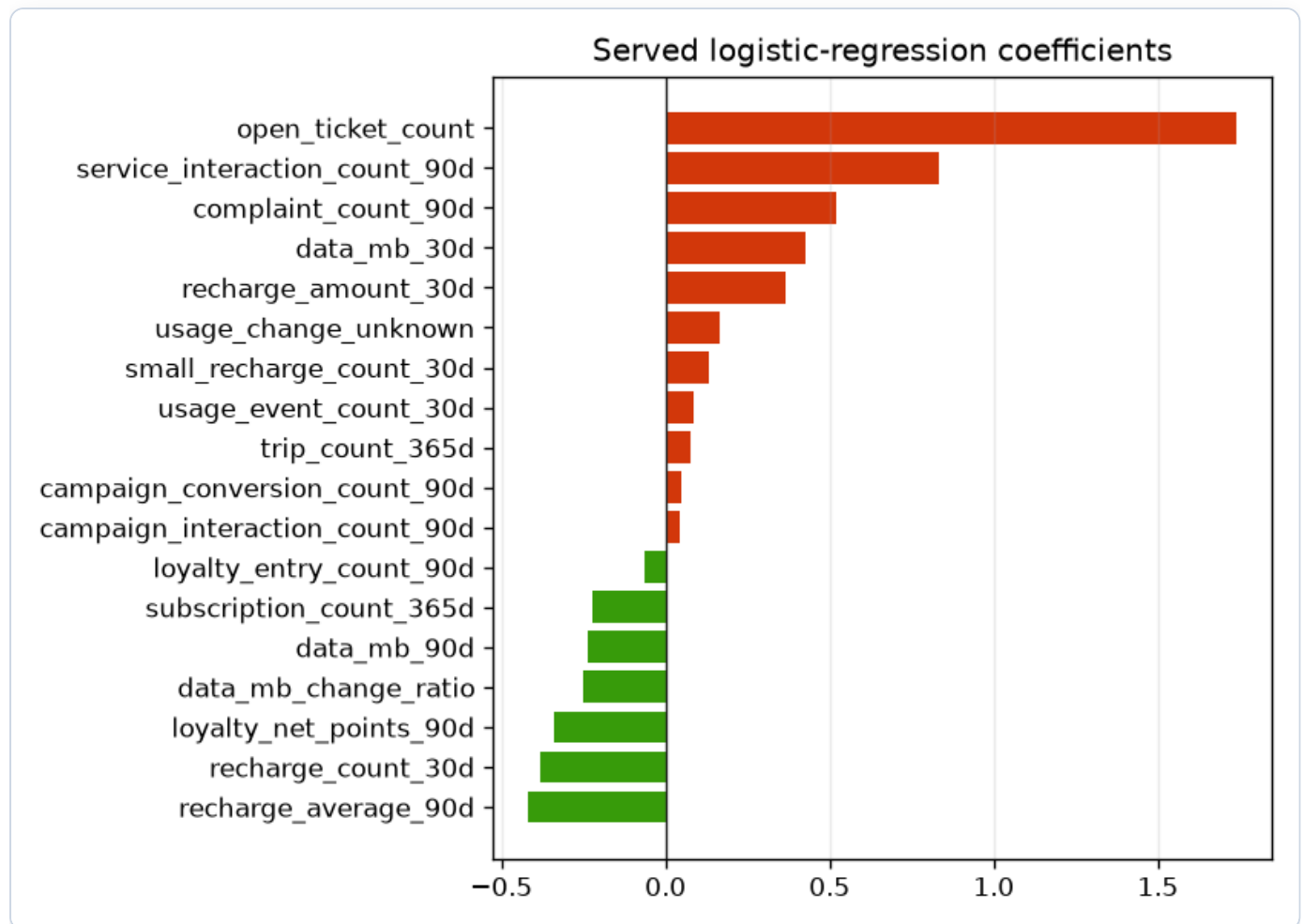


Notebook evidence

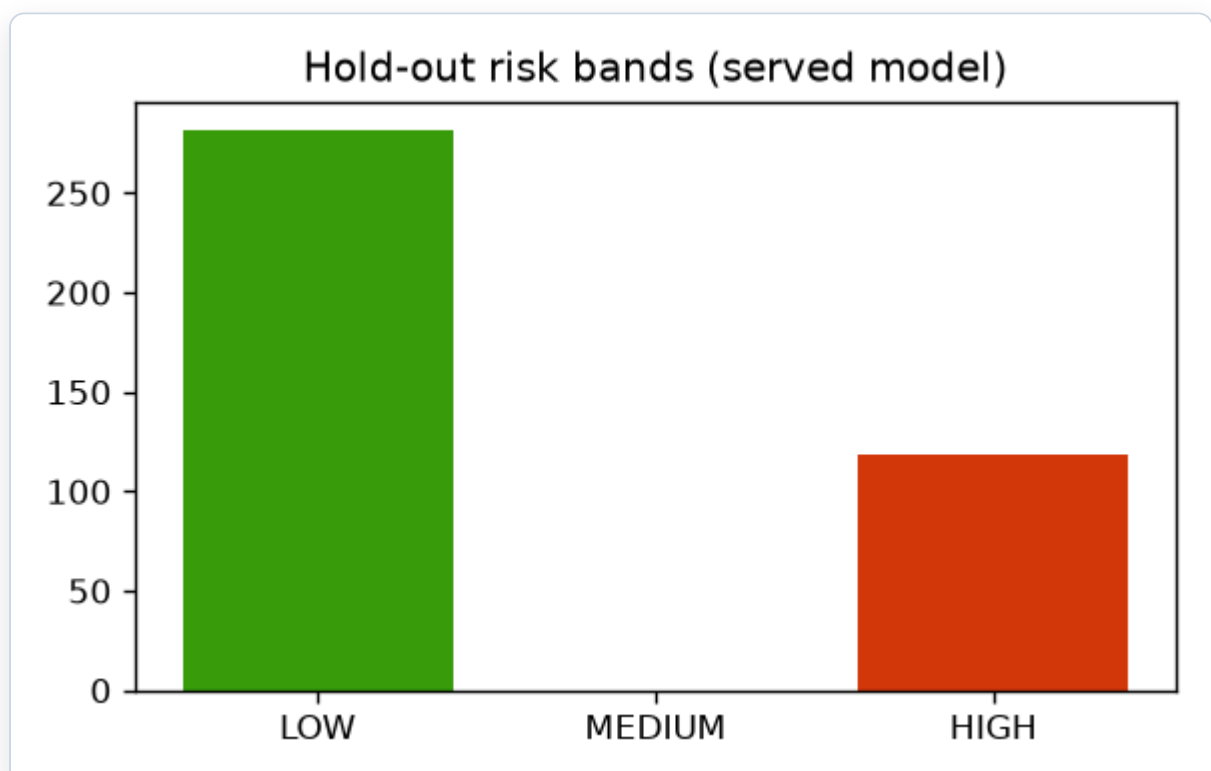
Hold-out model comparison



Interpretable logistic-regression drivers



Golden-customer risk bands



Supporting artifacts

- Churn metrics
- Model comparison table
- Logistic coefficients
- Golden customer scores
- Served model artifact

Technical details to present

Model comparison

Two supervised approaches were compared:

- Logistic regression
- Gradient-boosted trees

The selected model was logistic regression because it performed slightly better in the recorded hold-out comparison and remained simpler to explain and serve.

Evidence metrics

Model	ROC-AUC	PR-AUC	Brier score	Log loss
Logistic regression	0.9348	0.9145	0.0531	0.2281
Gradient boosting	0.9178	0.8992	0.0564	0.2525

Runtime artifact

The runtime uses exported coefficients and scaling parameters rather than loading the training framework. This reduces runtime dependencies and makes the scoring implementation transparent.

Output contract

The churn result includes:

- Probability
- Risk band
- Primary drivers
- Model version
- Feature snapshot
- `as_of`

Risk bands

- High: probability at or above 0.60
- Medium: probability at or above 0.35 and below 0.60

- Low: below 0.35

Example

U004 receives approximately 0.995 churn probability and a HIGH risk band.

Important limitation

The labels are generated from synthetic personas. The metrics prove the model-training and serving pipeline, not real-world churn accuracy or calibration.

Strong interview statement

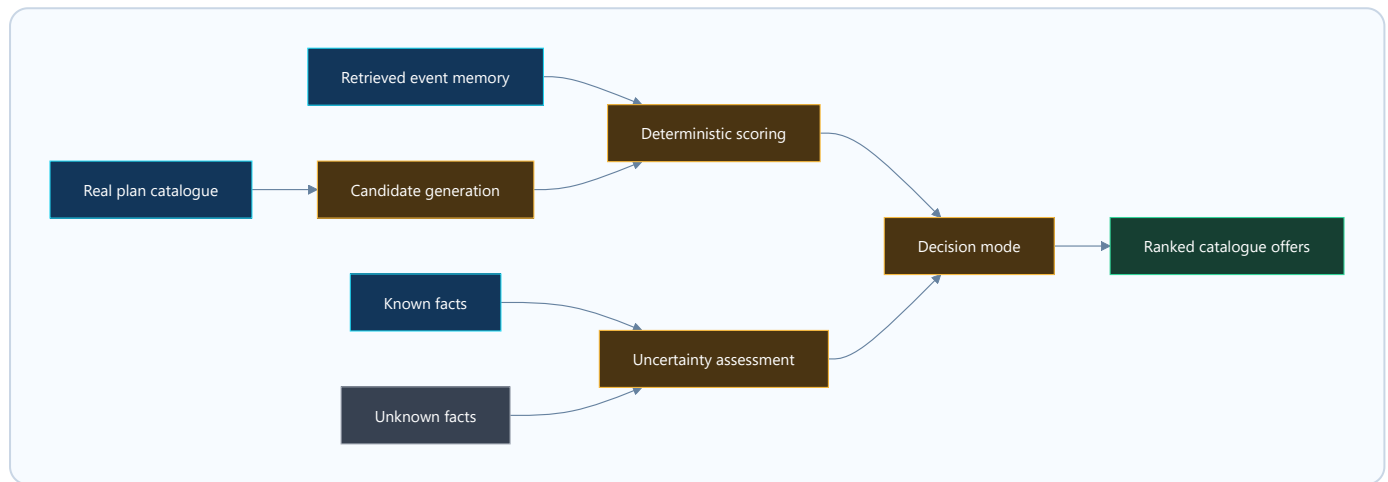
"I selected the model using both performance and operational simplicity. The runtime result is versioned and tied back to the feature snapshot that produced it."

Slide 9 — Capability 06: recommendations and uncertainty

Core message

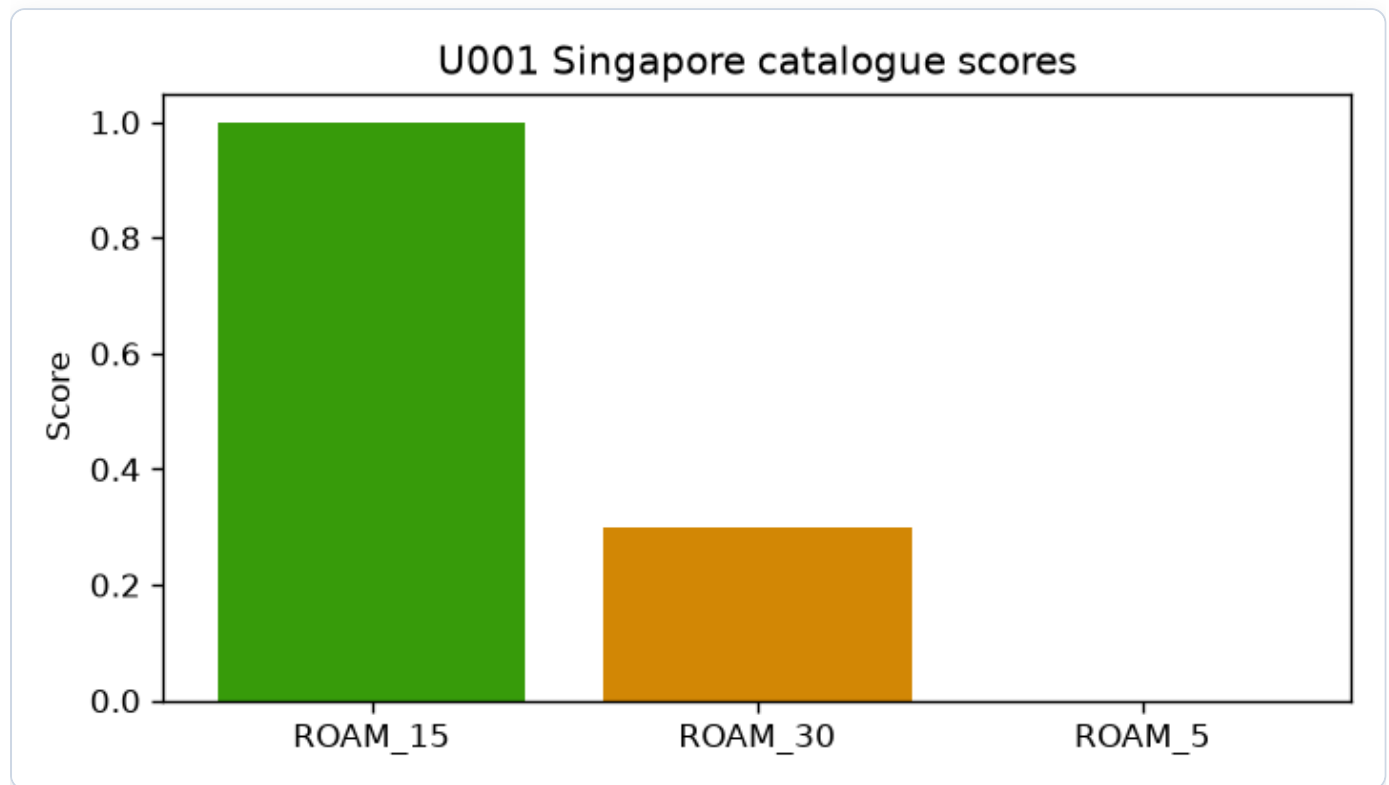
The recommendation service ranks only valid catalogue products and changes its decision mode when important information is unknown.

Explanation flowchart

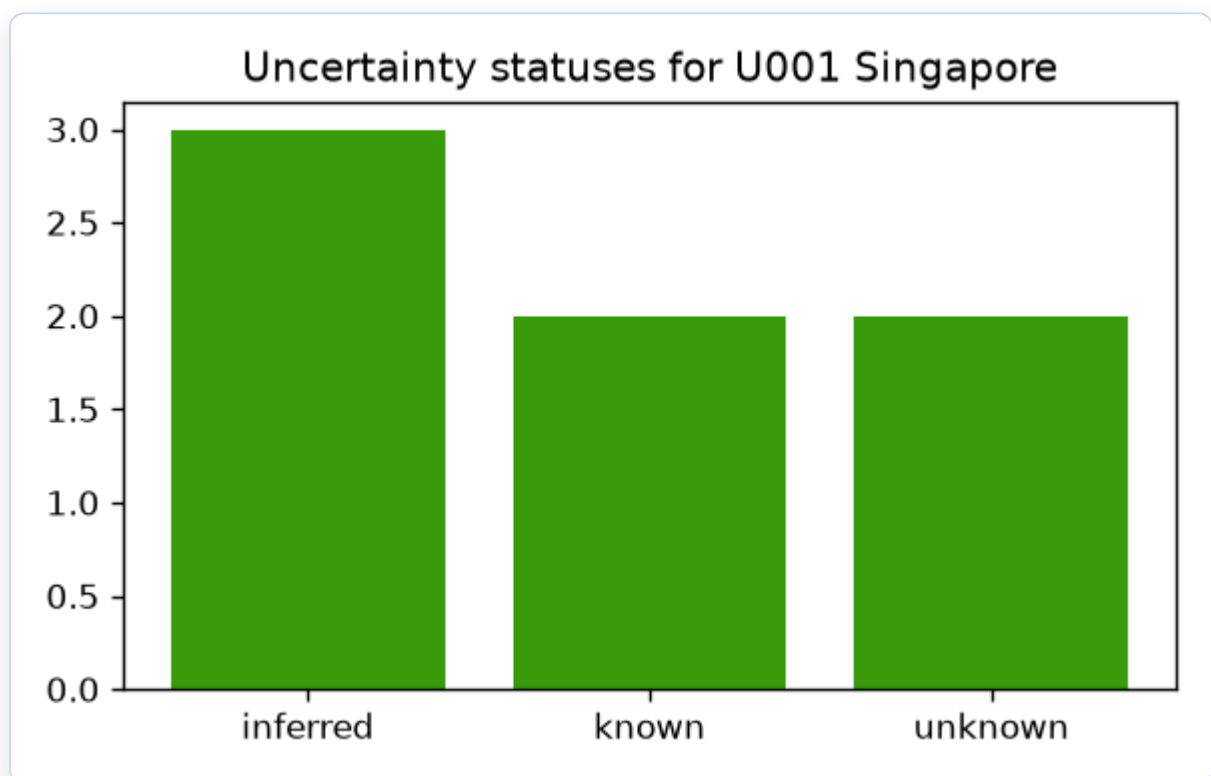


Notebook evidence

Catalogue candidate scores



Known, inferred, and unknown inputs



Supporting artifacts

- [Recommendation metrics](#)

- U001 ranked offers
- Decision modes
- Invented-plan rejection proof

Technical details to present

Candidate generation

Candidates come from the real plan catalogue. A model or language model cannot invent a product identifier.

Evidence sources

Recommendation scoring can use:

- Current destination
- Current duration when known
- Historical trip duration
- Previous plan
- Previous usage
- Previous outcome
- Catalogue eligibility and scenario fit

Uncertainty categories

Evidence is classified as:

- Known
- Inferred
- Predicted
- Unknown

Decision modes

- Single recommendation
- Ranked options
- Scenario based
- Ask for information
- No recommendation

Example scenario

For U001:

- Destination is known
- Current trip duration is unknown
- Historical duration, plan, and usage are inferred from memory
- Decision mode is `SCENARIO_BASED`

- ROAM_15 is ranked first
- ROAM_30 and ROAM_5 remain alternatives

Why scenario mode matters

Without current duration, returning one exact plan with false certainty would hide risk. Scenario mode presents options based on plausible duration ranges.

Current limitation

Ranking is deterministic, not a learned ranker. Outcome feedback and acceptance tracking are not yet part of this capability.

Strong interview statement

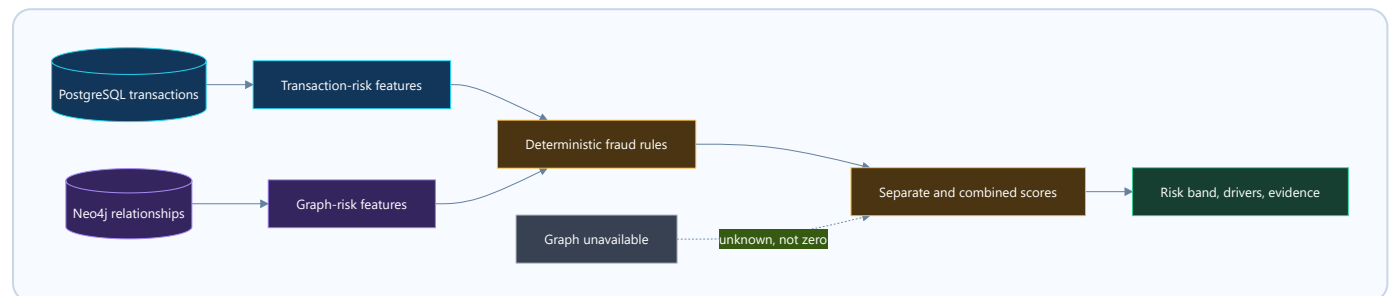
“The system never invents a plan, and it does not hide missing information. Unknown duration changes the response from an exact answer to scenario-based options.”

Slide 10 — Capability 07: graph fraud

Core message

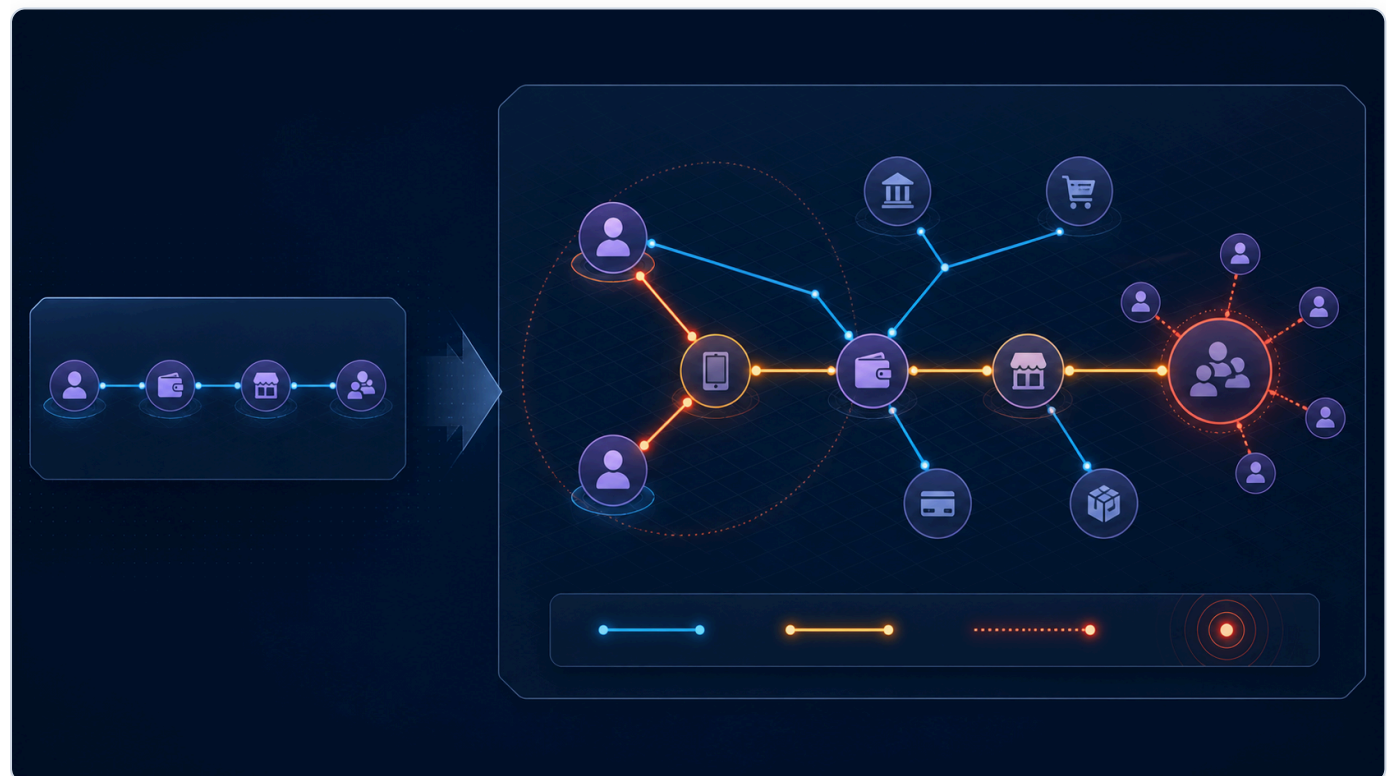
Graph context exposes connected risk that transaction-only analysis may miss.

Explanation flowchart

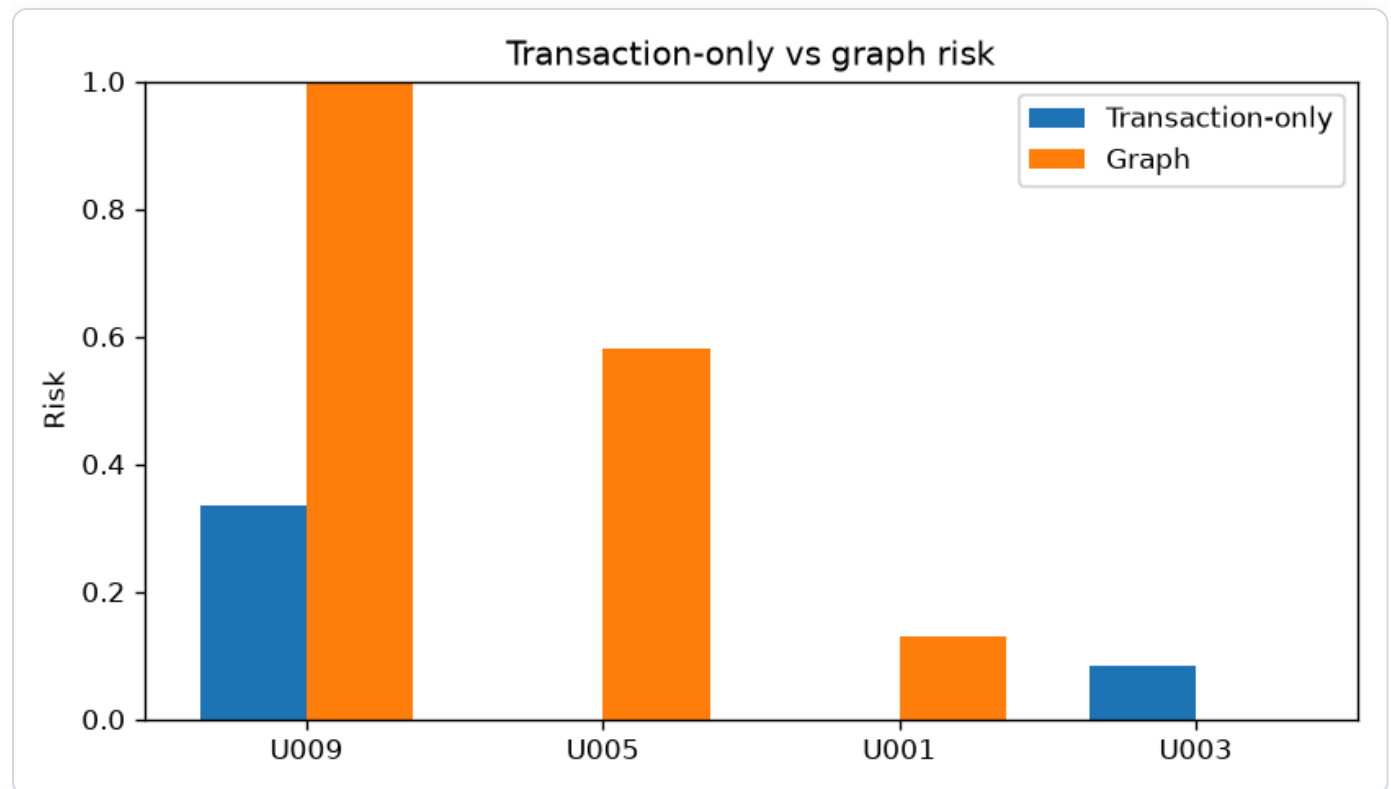


Concept, notebook, and UI evidence

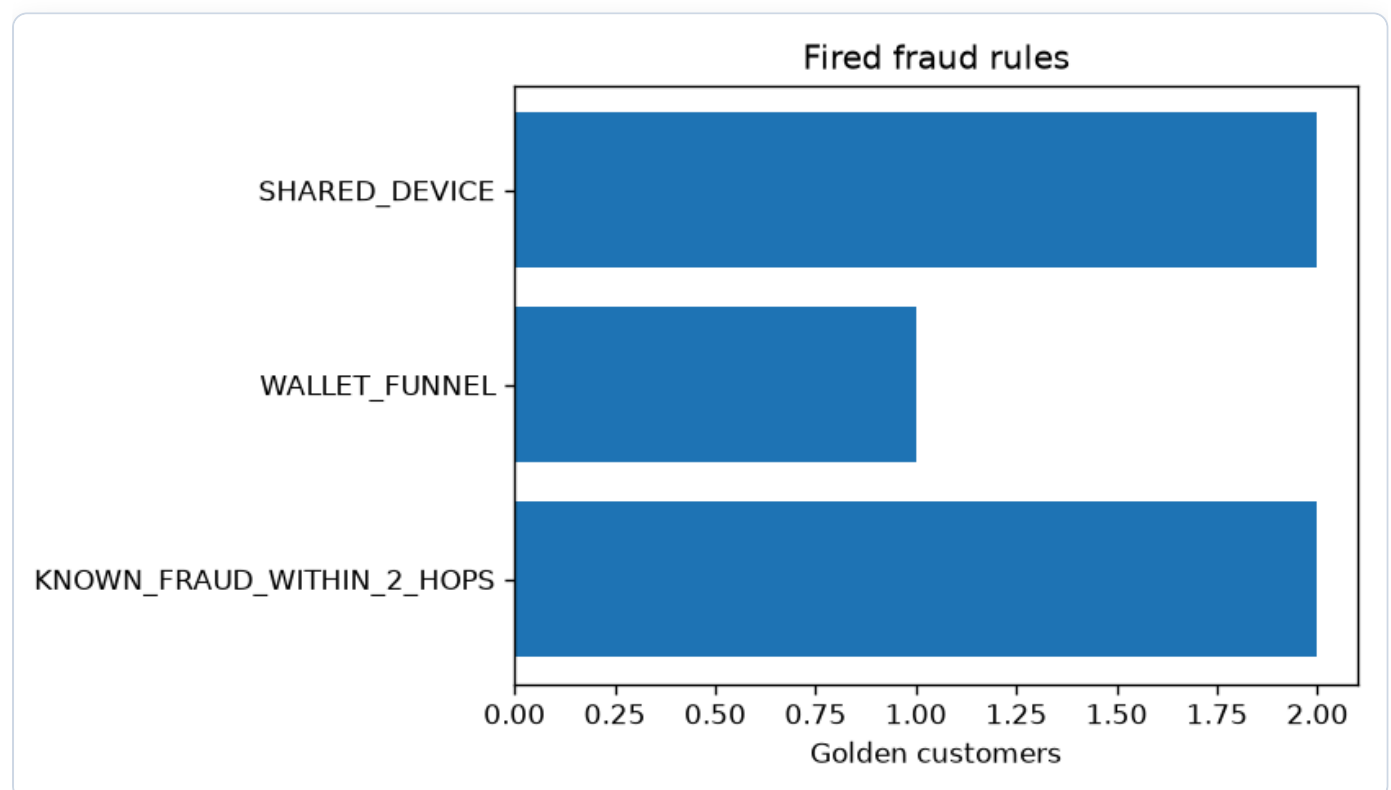
Relationship-context concept



Transaction-only versus graph risk



Deterministic rule firings



Supporting artifacts

- Fraud metrics
- Transaction-versus-graph data

- U009 fired rules
- Golden customer fraud scores

Technical details to present

Two independent evidence sources

Transaction risk uses PostgreSQL money activity, such as:

- Transaction amount and velocity
- Counterparty activity
- Account or wallet creation timing
- Circular movement visible in recorded transfers

Graph risk uses Neo4j relationships, such as:

- Shared devices
- Suspicious neighbours
- Wallet funnels
- Bounded distance to known fraud
- Connected merchant or wallet patterns

Implemented deterministic rules

- Shared device
- Known fraud within two wallet/transfer hops
- Wallet funnel
- Circular transfers
- Abnormal creation
- Abnormal transaction velocity

Important modelling detail

Shared-device risk is evaluated separately from wallet-based known-fraud distance. This prevents a shared device alone from being incorrectly interpreted as a wallet-transfer fraud path.

Score transparency

The result preserves:

- Transaction-only risk
- Graph risk
- Combined risk
- Fired rules
- Drivers
- Unknowns

Evidence scenario

For U009:

- Transaction-only risk: 0.335
- Graph risk: 1.0
- Final risk band: HIGH
- Graph risk exceeds transaction-only risk: true

Graph-unavailable behaviour

If Neo4j is unavailable, graph evidence remains unknown. The system does not report zero graph risk because zero would incorrectly imply that the relationships were observed and safe.

Current limitation

The POC does not block transactions, create a review queue, or serve graph embeddings. It demonstrates scoring and evidence only.

Strong interview statement

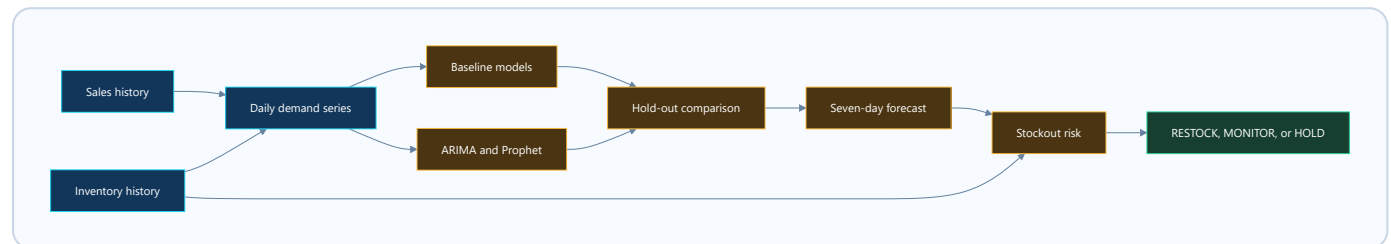
"The graph is valuable because it changes the evidence. U009 looks only moderately risky from its own transactions, but its connected neighbourhood produces maximum graph risk."

Slide 11 — Capability 08: SFA forecasting

Core message

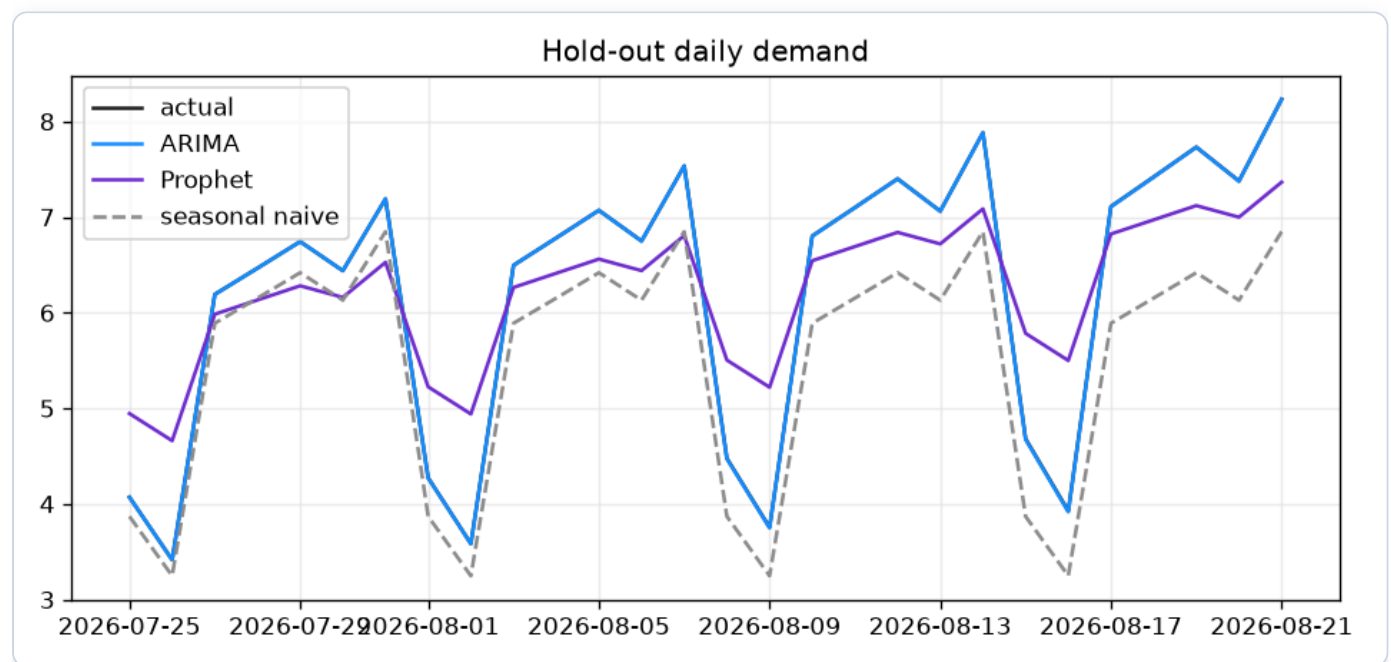
Retail demand is forecast from authoritative sales and inventory facts, then translated into stockout risk and a proposed action.

Explanation flowchart

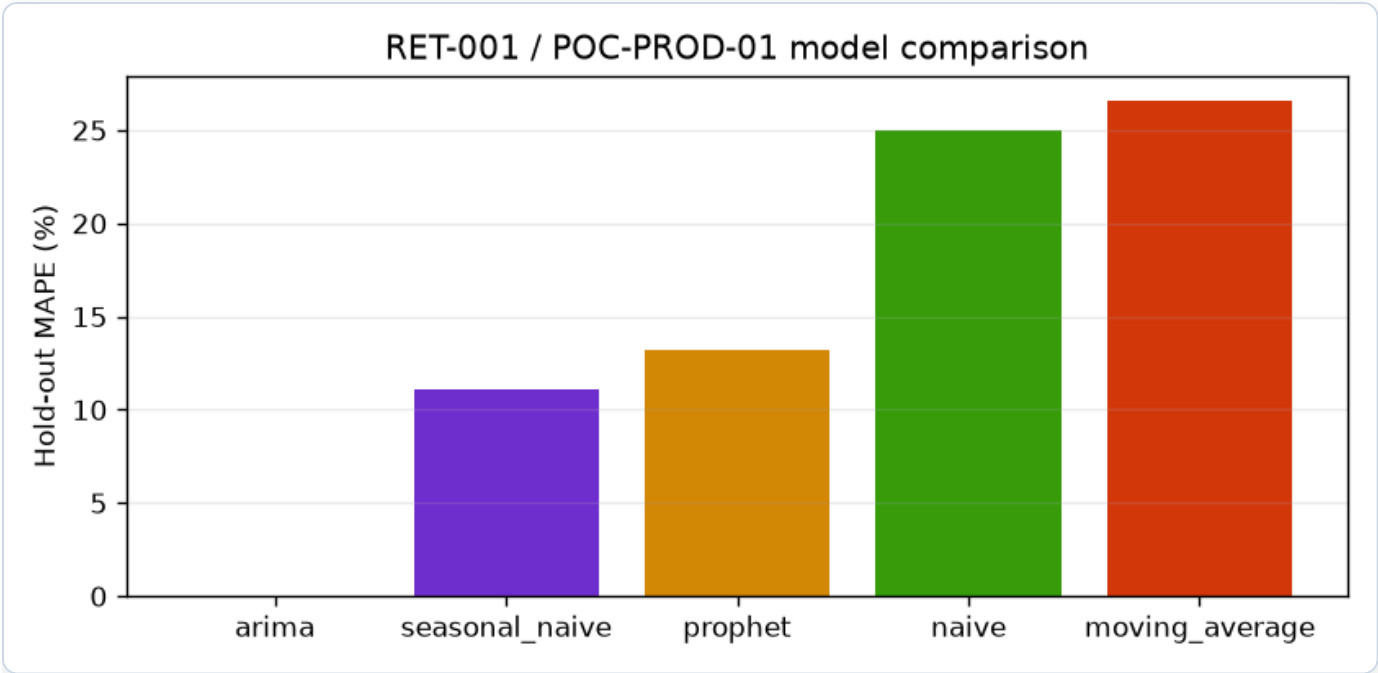


Notebook evidence

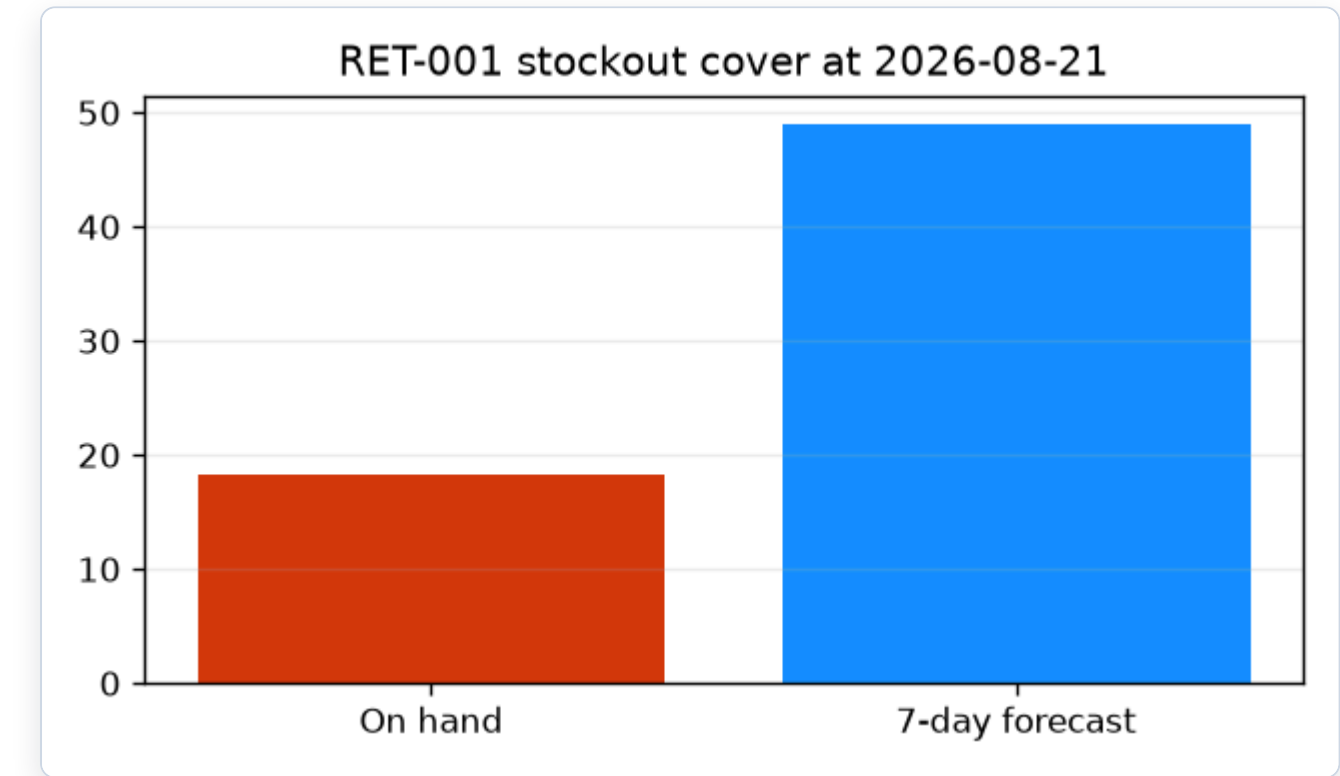
Hold-out demand forecast



Forecast-model comparison



Stockout cover



Supporting artifacts

- [Forecast metrics](#)
- [Hero forecast data](#)
- [Model comparison table](#)
- [Reconstruction evidence](#)
- [Served forecast artifact](#)

Technical details to present

Time-series inputs

- Historical retailer sales
- Product-level inventory observations
- Daily demand series derived from the recorded POC facts
- Explicit forecast `as_of`

Models compared

- Naïve
- Seasonal naïve
- Moving average
- ARIMA/SARIMAX
- Prophet

Model-selection evidence

Recorded hold-out MAPE:

- Naïve: 25.0031%
- Seasonal naïve: 11.1372%
- Moving average: 26.6045%
- ARIMA: 0.0367%
- Prophet: 13.1731%

ARIMA is the served model in the POC artifact.

Runtime output

- Seven-day demand forecast
- On-hand inventory
- Inventory cover
- Stockout probability
- Risk band
- Proposed action: `RESTOCK`, `MONITOR`, or `HOLD`

Evidence scenario

For retailer `RET-001` and product `POC-PROD-01`:

- On hand: 18.26
- Seven-day forecast: 49.06
- Seven-day actual: 47.34
- Stockout warning: true

Prediction versus action

The time-series model predicts demand. A separate rule uses forecast and inventory cover to propose an operational action.

Important limitation

The POC expands monthly pulses into daily demand, which makes the series more regular than real retail data. Supplier lead time, promotion effects, and operational constraints require more complete production data.

Strong interview statement

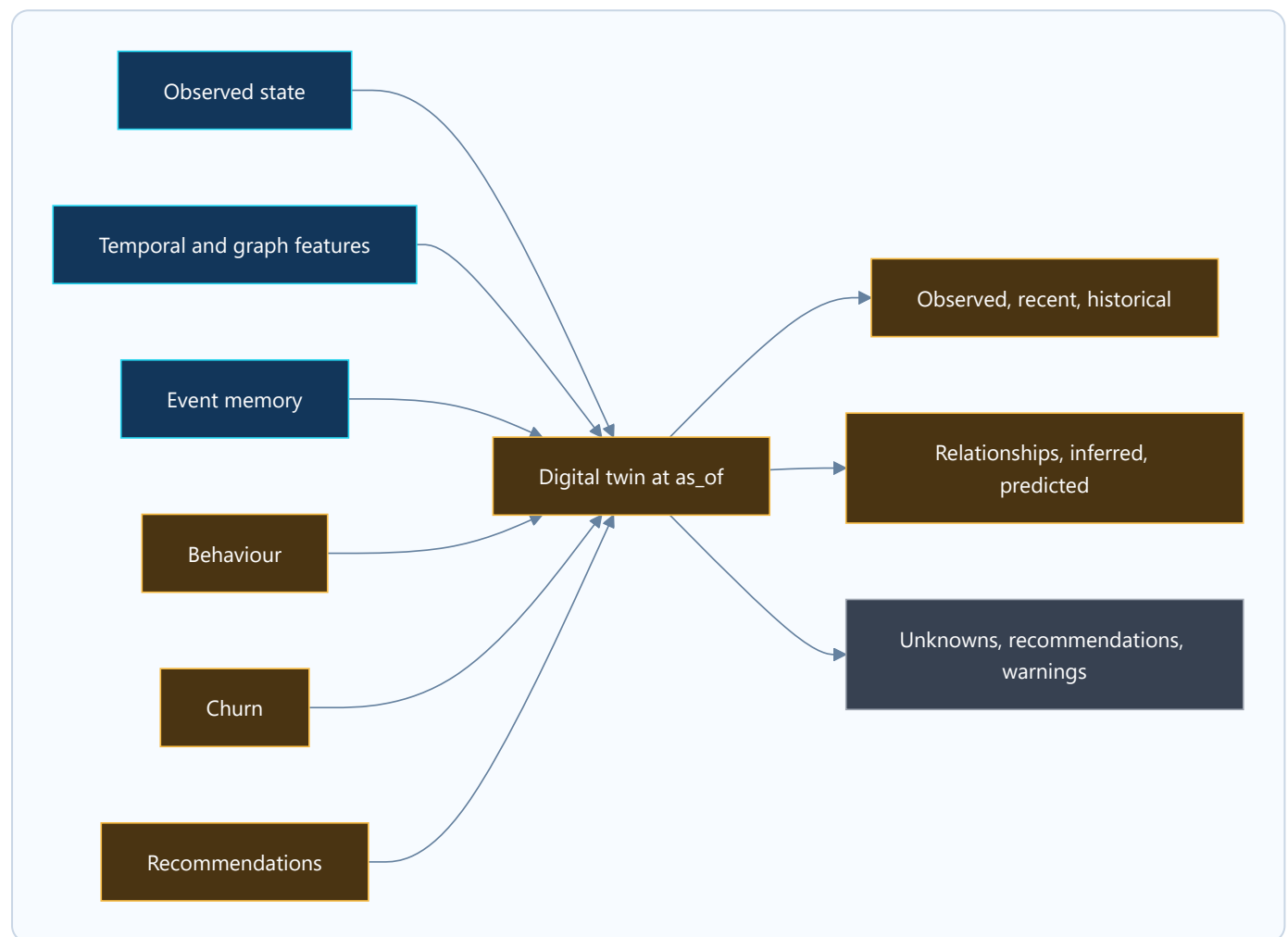
"I compared simple baselines before serving a more complex forecast. The forecast remains separate from the rule that proposes restocking."

Slide 12 — Capability 09: computed digital twins

Core message

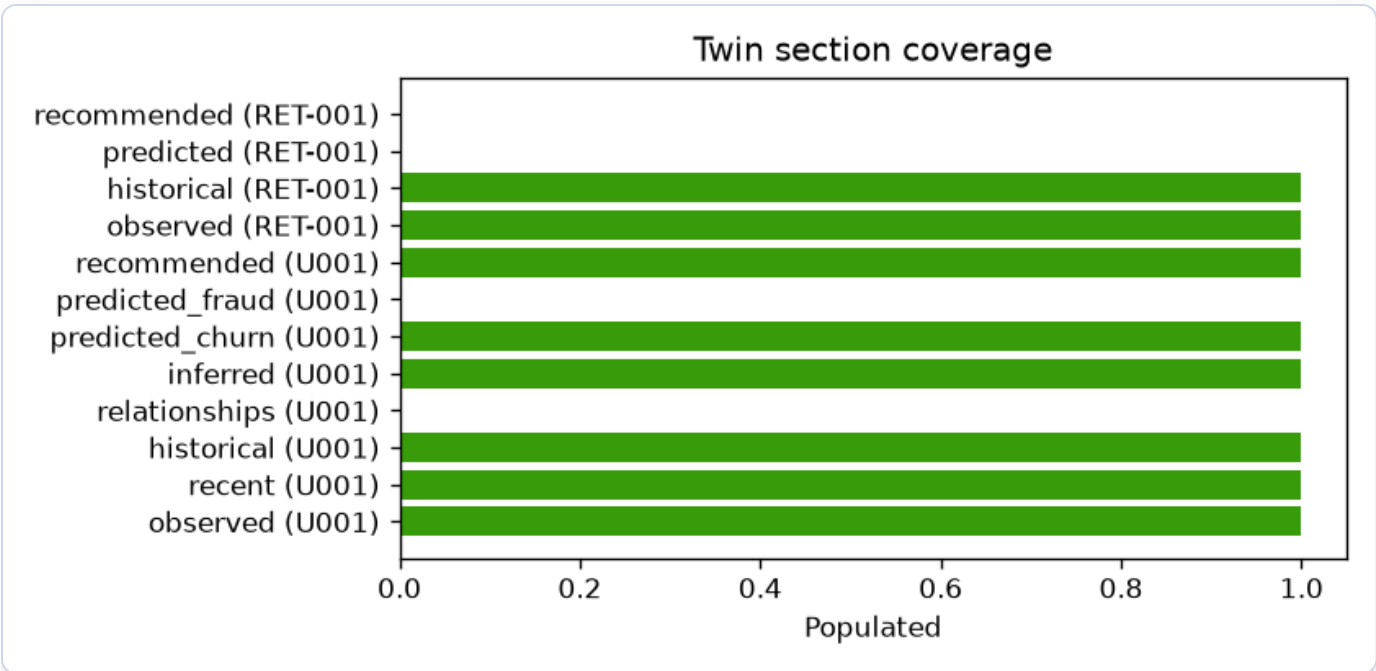
The digital twin assembles a complete point-in-time view without becoming another source of truth.

Explanation flowchart

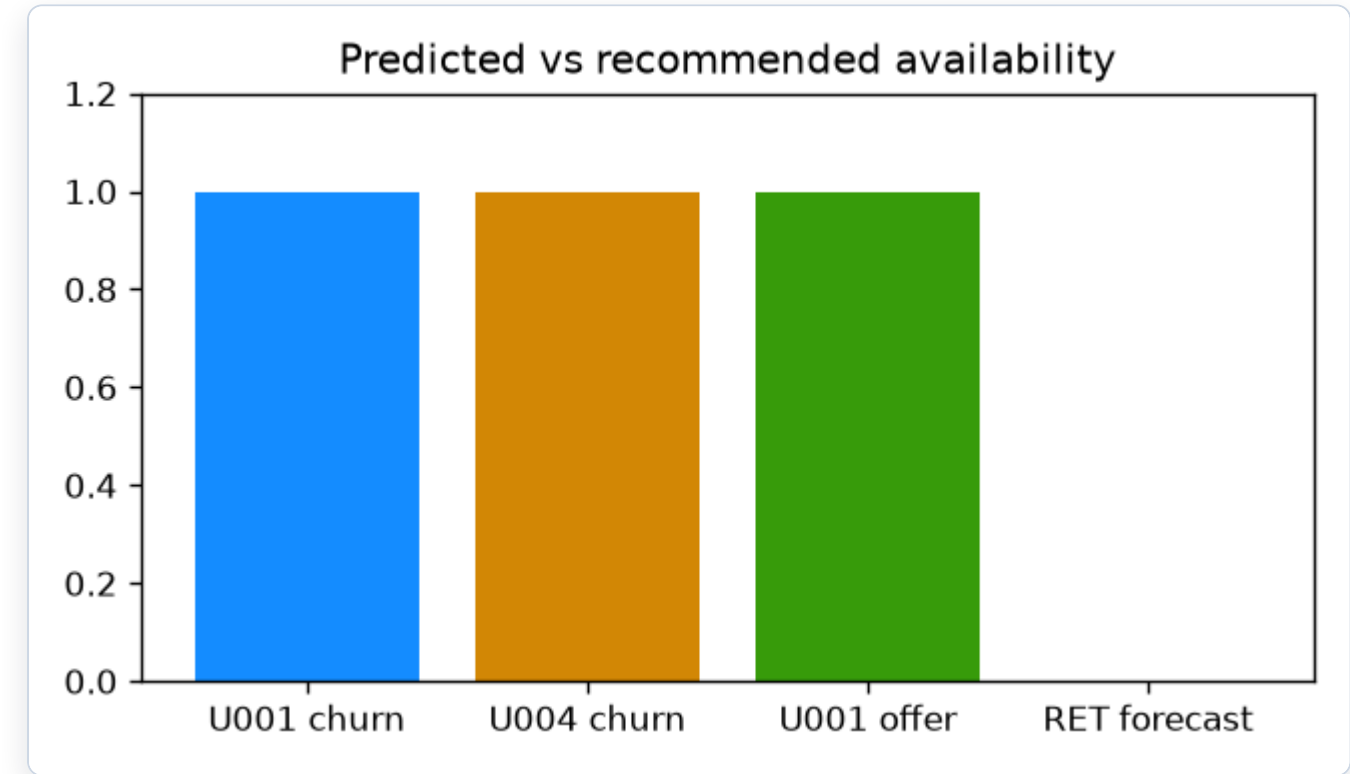


Notebook and UI evidence

Twin section coverage



Predicted versus recommended sections



Customer 360 twin experience

Customer 360
Recorded facts plus derived features, episodes, behaviour traits, a trained churn score, catalogue recommendations, graph fraud risk, a computed digital twin and a next-best action.

Golden persona: Frequent Traveller — U001
As of date (optional): mm/dd/yyyy
Application lens: All recorded facts
[Load facts](#)

U001 Golden persona

FREQUENT_TRAVELLER • PREPAID • LK
Plan: Roaming 15GB • Balance: 1350.0000 LKR
Source: live_database • as_of 31 Aug 2026, 23:59 UTC

Usage	Recorded fact
16 Mar 2026, 12:00 UTC STREAMING 900.0000 MB in SG Source: live_database • as_of 31 Aug 2026, 23:59 UTC • telco:usage_event	Recorded fact
15 Mar 2026, 20:00 UTC STREAMING 1400.0000 MB in SG Source: live_database • as_of 31 Aug 2026, 23:59 UTC • telco:usage_event	Recorded fact
14 Mar 2026, 20:00 UTC STREAMING 1600.0000 MB in SG Source: live_database • as_of 31 Aug 2026, 23:59 UTC • telco:usage_event	Recorded fact
13 Mar 2026, 20:00 UTC STREAMING 1700.0000 MB in SG Source: live_database • as_of 31 Aug 2026, 23:59 UTC • telco:usage_event	Recorded fact
12 Mar 2026, 20:00 UTC STREAMING 1900.0000 MB in SG Source: live_database • as_of 31 Aug 2026, 23:59 UTC • telco:usage_event	Recorded fact
11 Mar 2026, 20:00 UTC STREAMING 2100.0000 MB in SG Source: live_database • as_of 31 Aug 2026, 23:59 UTC • telco:usage_event	Recorded fact
10 Mar 2026, 20:00 UTC STREAMING 1800.0000 MB in SG Source: live_database • as_of 31 Aug 2026, 23:59 UTC •	Recorded fact

Recharge history	Recorded fact
1 Mar 2026, 08:00 UTC 2000.0000 LKR via APP Source: live_database • as_of 31 Aug 2026, 23:59 UTC • telco:recharge	Recorded fact

Derived features

customer-features-v1 • as_of 31 Aug 2026, 23:59 UTC

usage (30d)

Event Count 30d	0
Event Count 90d	0
Data Mb 30d	0
Data Mb 90d	0
Data Mb Previous 30d	0
Data Mb Change Ratio	Unknown

recharge (30d)

Count 30d	0
Amount 30d	0
Average 90d	Unknown
Small Recharge Count 30d	0
Frequent Small Recharge Evidence	

money (90d)

Supporting artifacts

- Digital-twin metrics
- U001 customer twin
- Retailer twin
- Section coverage data

Technical details to present

Customer-twin sections

- Observed
- Recent
- Historical
- Relationships
- Inferred
- Predicted
- Unknown
- Recommended
- Warnings

Customer composition

The customer twin combines:

- Observed customer state
- Temporal and graph features

- Retrieved historical episodes
- Behaviour traits
- Churn prediction
- Recommendations

Retailer twin

The retailer twin provides:

- Observed facts
- Historical activity
- Predicted values where available
- Recommended actions where available

Unknown-first design

Missing sections remain visible rather than receiving empty default values. Examples include:

- Neo4j graph unavailable
- Current trip duration unknown
- Previous feature window empty
- Capability not included in the current twin contract

Evidence

- U001 primary offer: ROAM_15
- U001 traits: heavy data user, frequent traveller, streaming heavy
- U004 churn band: HIGH
- Relationship and time-window unknowns remain explicit

Bounded composition

Graph fraud and SFA forecasting are live as separate panels. They are not silently injected into a twin contract that was defined to compose a smaller capability set.

Strong interview statement

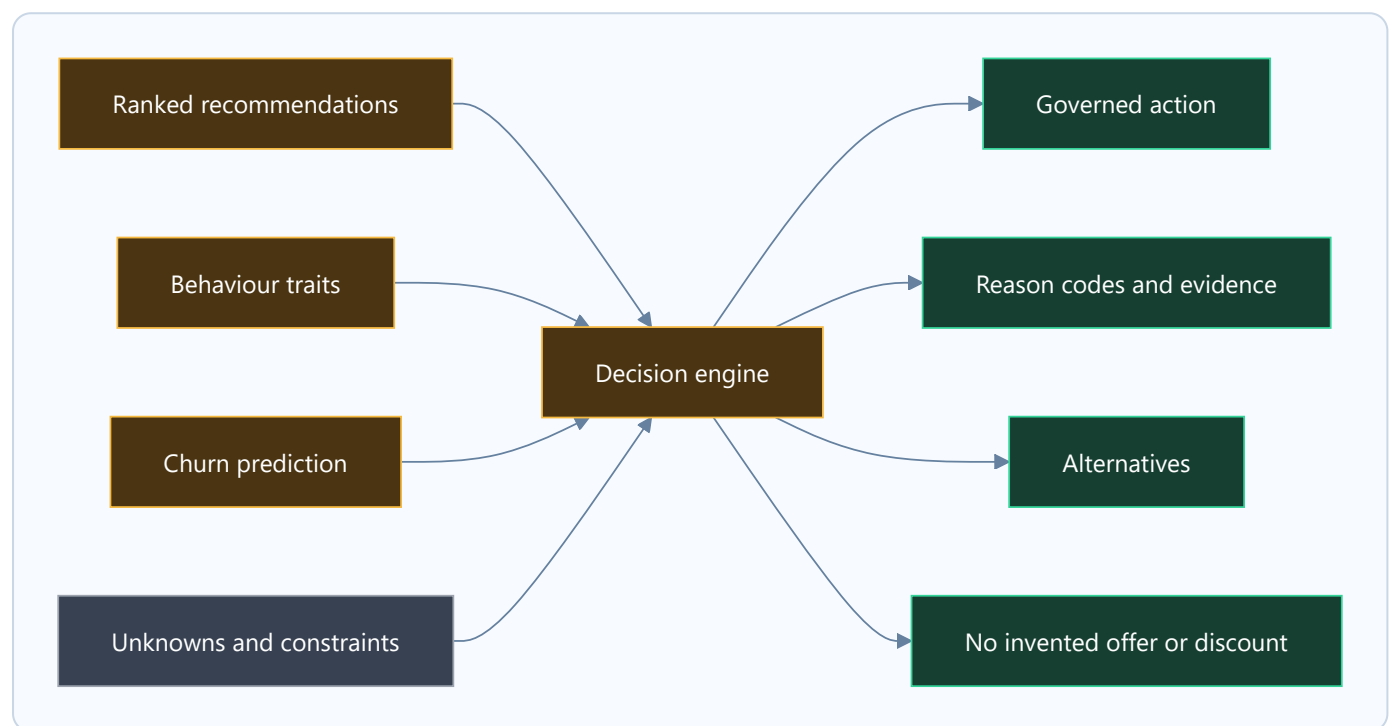
"The twin is not a giant customer table. It is a computed, typed view that tells the consumer what is observed, inferred, predicted, recommended, or still unknown."

Slide 13 — Capability 10: decision engine and explanations

Core message

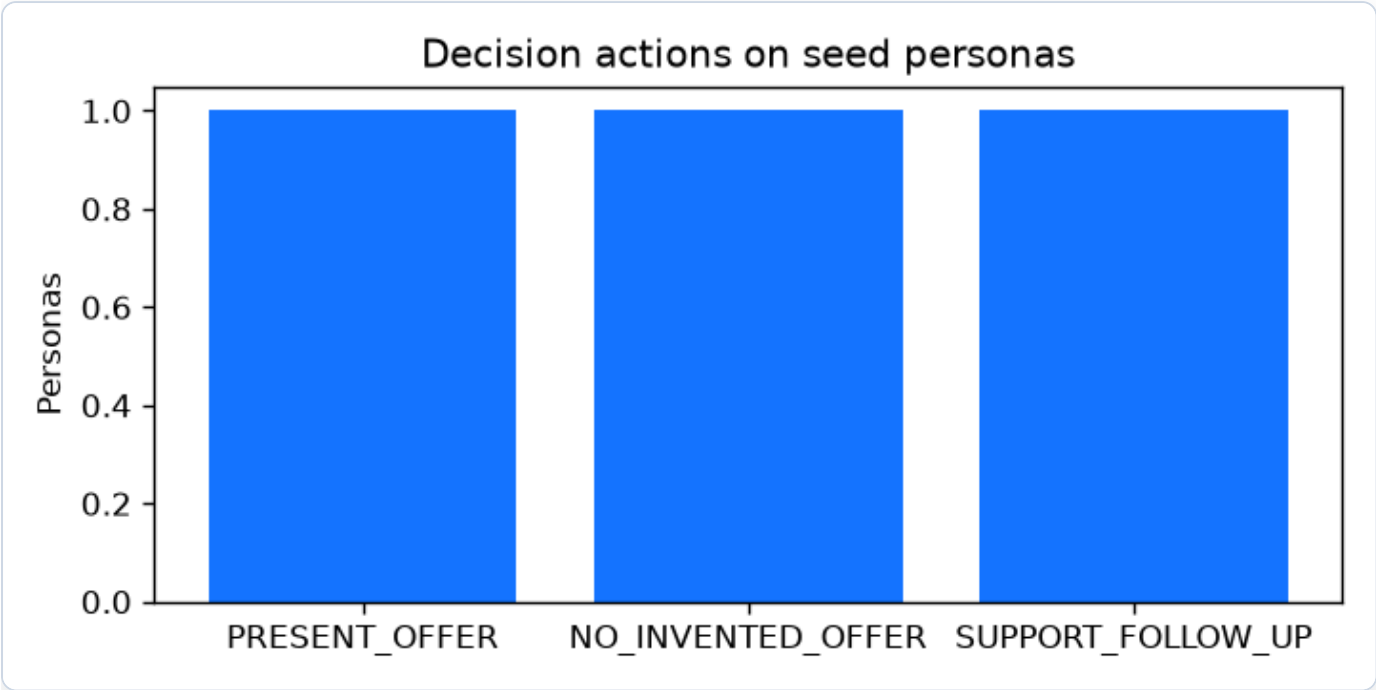
Predictions inform decisions, but they do not directly become business actions.

Explanation flowchart



Notebook and UI evidence

Decision actions across seed personas



Live next-best-action view

INTELLIGENCE

Overview

Customer 360

Journey and Event Memory

Campaigns and Loyalty

Money and Fraud

Retail and SFA

Graph Explorer

Models and Decisions

Copilot

POC Status

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Live evidence

Models and Decisions

Live next-best actions from the decision engine. Computed twins are live on Customer 360.

Golden persona

As of date (optional)

Query destination

Frequent Traveller — U001

mm/dd/yyyy

SG

Load next-best action

Decision

Recommend

PRESENT OFFER • ROAM_15

What

Why

Alternatives

HISTORICAL EPISODE CATALOGUE MATCH DURATION UNKNOWN

customer-decision-v1

Digital twins

Live evidence

Computed twins are live on Customer 360. This page scores next-best actions only.

Open Customer 360 twin

Live evidence

Present catalogue offer ROAM_15.

Retrieved travel memory ranks ROAM_15 for the 4-7 days scenario.

POC_ROAM_15, ROAM_30, POC_ROAM_7, POC_ROAM_3, ROAM_5

Supporting artifacts

- Decision metrics
- Persona decisions
- No-invented-discount proof

Technical details to present

Inputs

The current decision engine composes:

- Ranked recommendations
- Behaviour traits
- Churn prediction

Supported actions

- PRESENT_OFFER
- SUPPORT_FOLLOW_UP
- REQUEST_INFORMATION
- NO_INVENTED_OFFER

Governance behaviour

- High churn can block an upsell.
- High churn cannot create an arbitrary discount.
- A recommendation must refer to a real catalogue plan.
- Missing information can trigger a request for information.
- No valid candidate produces no invented offer.

Evidence scenarios

- U001: present ROAM_15
- U004: support follow-up because churn risk blocks normal upsell behaviour
- U002: no invented offer

Explanation structure

Every decision can expose:

- What action was selected
- Why it was selected
- Evidence and reason codes
- Confidence or uncertainty
- Unknown inputs
- Alternative actions or offers

Prediction versus decision

Prediction: what may happen?

Decision: what should the business do under rules and constraints?

Current limitation

The engine composes a defined subset of capabilities. Graph fraud, forecasting, and twins remain separate outputs rather than being silently added to the decision logic.

Strong interview statement

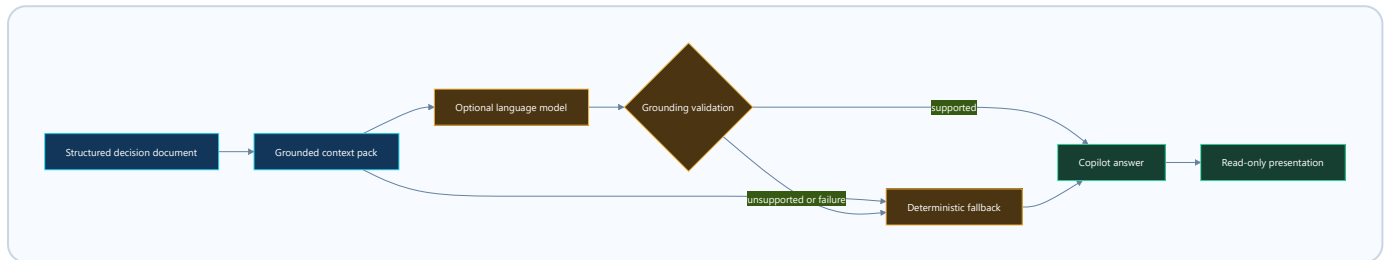
"A 99% churn probability is not a business action. The decision engine interprets that prediction under catalogue, eligibility, uncertainty, and customer-support rules."

Slide 14 — Capability 11: grounded Copilot

Core message

Copilot explains structured intelligence. It does not calculate risk, invent products, or execute business commands.

Explanation flowchart



Notebook and UI evidence

Live grounded Copilot

COMPANY INTELLIGENCE

NG Application Shell • Company Intelligence POC

INTELLIGENCE

Overview

Customer 360

Journey and Event Memory

Campaigns and Loyalty

Money and Fraud

Retail and SFA

Graph Explorer

Models and Decisions

Copilot

POC Status

Copilot

Read-only answers from the decision document. The model cannot invent plans, discounts or destinations.

Golden persona

Frequent Traveller — U001

As of date (optional)

mm/dd/yyyy

Query destination

SG

Question

Why is U001 receiving this recommendation?

Ask Copilot

Answer

Model

Why U001 is receiving this recommendation:

Action:

Present catalogue offer

Reason codes:

HISTORICAL_EPISODE, CATALOGUE_MATCH, DURATION_UNKNOWN

The explanation:

1.

Historical travel memory:

A retrieved travel episode from U001's history shows prior use of the same plan (ROAM_15, with 11.4 GB of historical usage), which ranks ROAM_15 for the 4–7 days scenario.

2.

Catalogue match:

ROAM_15 is a valid catalogue plan on the allowed list.

3.

Scenario-based mode:

Because U001's trip duration is unknown, offers are ranked as duration scenarios rather than an exact-duration match — this is flagged by the DURATION_UNKNOWN reason code.

4.

Supporting traits:

U001 is profiled as a FREQUENT_TRAVELLER, HEAVY_DATA_USER, and STREAMING_HEAVY customer, with a LOW churn risk band (probability ≈ 0.0893).

Alternatives considered:

POC_ROAM_15, ROAM_30, POC_ROAM_7, POC_ROAM_3, ROAM_5

Note:

The chosen offer and its outcome are not recorded in this capability, so whether U001 accepts or benefits from ROAM_15 is unknown here.

Source: openrouter_glm • as_of 31 Aug 2026, 23:59 UTC

customer-copilot-v1 • z-ai/glm-5.3-flash • grounded

Provenance

Derived

action=PRESENT_OFFER

target=ROAM_15

Supporting artifacts

- Copilot metrics
- Grounded deterministic fallback

The evidence confirms that the fallback names `ROAM_15`, explains that trip duration is unknown, and does not mention a fabricated plan.

Technical details to present

Context construction

The Copilot receives a structured decision document containing:

- Selected action
- Target plan when applicable
- Reason codes
- Historical evidence
- Alternatives
- Churn band
- Known and unknown information

Two response paths

1. Deterministic fallback response
2. Optional external language-model response

The deterministic response is always available.

Grounding validation

The system rejects responses that introduce unsupported content, such as:

- A plan outside the catalogue context
- An invented discount
- A destination not present in context
- A reason not supported by the decision document

Provider-failure behaviour

If the external model is unavailable, misconfigured, or returns ungrounded text, the platform returns the deterministic fallback instead of failing the entire user experience.

Evidence

- Notebook response source: deterministic fallback
- `ROAM_15` mentioned: true
- Unknown trip duration mentioned: true
- Fake plan mentioned: false

Security boundary

- Provider keys remain in environment configuration.
- Keys are not placed in frontend files.
- Copilot is read-only.
- It does not have access to command execution.

- Conversation history is not implemented in the POC.

Strong interview statement

"The LLM is the last layer, not the intelligence engine. The facts, scores, recommendation, decision, and explanation already exist before the prompt is created."

Cross-capability technical themes

Point-in-time correctness

The same `as_of` principle applies to:

- State reconstruction
- Temporal features
- Graph features
- Event memory
- Behaviour
- Churn
- Recommendations
- Fraud
- Forecasting
- Digital twins
- Decisions

This prevents future leakage and supports reproducibility.

Explicit unknowns

The platform distinguishes:

- Observed zero
- Missing data
- Unavailable dependency
- Capability outside the current contract
- Information that is not yet known at `as_of`

This is especially important for graph availability, trip duration, previous feature windows, and uncomposed twin sections.

Baseline-first modelling

The POC compares understandable baselines before adding complexity:

- Logistic regression versus gradient boosting for churn
- Naive and seasonal-naive forecasting before ARIMA and Prophet
- Deterministic fraud rules before graph ML
- Deterministic recommendation ranking before a learned ranker
- Deterministic Copilot fallback before optional LLM narration

Versioning and provenance

Important derived contracts include a version and provenance:

- Dataset version
- Feature-set version
- Episode-set version
- Behaviour-set version
- Churn-model version
- Fraud-scorer version
- Forecast-model version
- Decision version
- Copilot context version

This makes debugging and comparison possible when logic evolves.

Separation of responsibility

Layer	Responsibility
Data layer	Authoritative facts and immutable history
Graph projection	Rebuildable relationship context
Feature layer	Reusable point-in-time derived inputs
Intelligence layer	Traits, predictions, recommendations, and risk
Twin layer	Composed contextual view
Decision layer	Governed action selection
Explanation layer	Evidence, alternatives, and uncertainty
Copilot layer	Natural-language presentation

Honest POC limitations

These points should be stated clearly rather than hidden:

- The dataset is synthetic and scenario-shaped.
- POC model metrics do not prove production accuracy or calibration.
- Thresholds and rules are demonstrative.
- Neo4j projection uses a controlled snapshot approach.
- Distributed projection workers and dead-letter handling are not implemented.
- Production authentication and tenant isolation are not implemented.
- Continuous model monitoring, drift, and fairness processes are not implemented.
- Recommendation and decision outcome feedback is not complete.
- Fraud review queues and transaction blocking are not implemented.
- The Copilot is read-only and has no conversation history.
- The interactive browser simulator is not implemented.

The correct positioning is:

“The POC proves the architecture, contracts, scenarios, and evidence chain. Production readiness requires operational hardening and real outcome data.”

Questions to prepare for

Why PostgreSQL instead of putting everything in Neo4j?

PostgreSQL provides the authoritative transactions, ledgers, constraints, and historical event records. Neo4j is added for multi-hop relationship traversal. The systems have different responsibilities.

Why is the graph rebuildable?

It prevents the graph from becoming a second source of truth and enables recovery, reconciliation, and replay from authoritative facts.

How is future leakage prevented?

Every intelligence service accepts an explicit timezone-aware `as_of` and filters facts and episodes to information available at or before that time.

Why not let the ML model choose the action directly?

A model score does not encode catalogue validity, policy, eligibility, uncertainty, customer-support rules, or alternatives. Those belong in a decision layer.

Why use deterministic rules in several capabilities?

Rules create transparent baselines, are easy to validate, and demonstrate the evidence chain. They can later be compared with learned approaches without changing the platform contracts.

What is the most valuable graph use case?

Graph fraud, because shared devices, wallet paths, merchant funnels, circular movement, and proximity to known fraud can reveal risk not visible in an isolated transaction.

What would be the first production improvements?

1. Authentication and tenant isolation
2. Observability and projection-lag monitoring
3. Distributed outbox workers and dead-letter handling
4. Real outcome collection
5. Model calibration, drift, and fairness monitoring
6. Load and failure testing
7. Human review workflows for high-risk decisions

Recommended closing

“The main result is a reusable technical foundation. PostgreSQL preserves trusted facts and history, Neo4j adds relationship intelligence, point-in-time feature contracts support multiple capabilities, and a governed decision layer keeps predictions separate from business actions. The POC demonstrates the complete evidence chain while remaining honest about synthetic data and production gaps.”